FINAL PROJECT REPORT

A Domain-Specific Language and Compiler Front-End for Data Science Pipelines

BCSE307P - Compiler Design Laboratory

Fall Semester 2026-27

Phase 3 Final Implementation and Testing

Prepared by

SINCHANA B KURKURI

24BDS0352

Slot L49+L50

Abstract

Data science pipeline descriptions contain dependencies that can be checked before an execution engine is invoked. This project implements a domain-specific language and compiler front-end for expressing such descriptions as one named pipeline. The language supports dataset declarations, cleaning, column selection and dropping, percentage-based partitions, model declarations and evaluation requests. A regular-expression lexer produces tokens with source coordinates. A hand-written recursive-descent parser constructs a dataclass-based abstract syntax tree. Semantic analysis traverses the tree with a symbol table to enforce declaration, ordering, column and partition constraints. A lowering stage produces opcode-and-argument instructions. Four optimizer passes remove instructions after the final evaluation, eliminate unused training declarations, merge adjacent drops and collapse adjacent duplicate cleaning or selection instructions. A symbolic executor formats the optimized instructions as a readable trace; no dataset contents are read and no numerical model training or accuracy calculation is performed. A command-line interface exposes intermediate results, while a Streamlit dashboard presents the same compiler stages interactively. Validation records 267 passing tests and the outcomes of all 20 supplied example programs. Optimizer tests compare symbolic evaluation records, test transformation boundaries and check repeated optimization. Recorded benchmark measurements are examined by stage, time per statement and instruction reduction. The results demonstrate an inspectable compiler pipeline within a deliberately restricted language, with explicit limits on execution, schema validation and error recovery.

Table of Contents

Chapter 1 Introduction 1
1.1 Background and problem area 1
1.2 Motivation 1
1.3 Language and implementation overview 1
1.4 Report organization 1
Chapter 2 Problem Statement 2
2.1 Problem definition 2
2.2 Functional boundaries 2
2.3 Inputs outputs and acceptance conditions 2
Chapter 3 Objectives 3
3.1 Objectives carried forward from Phase 1 3
3.2 Scope decisions and additional outcomes 4
Chapter 4 Literature Survey or Background Study 5
4.1 Workflow orchestration and library pipelines 5
4.2 Compiler foundations 5
4.3 Gap addressed 6
Chapter 5 System Design 7
5.1 Architecture and data flow 7
5.2 Failure propagation 8
5.3 Module responsibilities 8
5.4 Symbol table and IR design 9
5.5 Interfaces and presentation design 10
5.6 Project directory organization 10
Chapter 6 Methodology 11
6.1 Phased development 11
6.2 Development history 11
6.3 Design decisions 11
6.4 Optimizer corrections and regression validation 12
6.5 Reconciliation of earlier descriptions 12
Chapter 7 Compiler Design Concepts Used 13
7.1 Lexical analysis 13

7.2 Syntax analysis....14
7.3 Abstract syntax tree....15
7.4 Symbol table and semantic analysis....16
7.5 Intermediate representation....19
7.6 Four-pass optimization....19
7.7 Symbolic execution....21
7.8 Error detection and handling....22
Chapter 8 Implementation....23
8.1 Language tools and versions....23
8.2 Source inventory....23
8.3 Core algorithms....24
8.4 Selected implementation excerpts....24
8.5 Command-line interface....26
8.6 Benchmark implementation....26
8.7 Dashboard implementation....26
Chapter 9 Testing and Results....28
9.1 Test strategy and executed results....28
9.2 Baseline compilation and drop integration....29
9.3 Representative expected and actual cases....30
9.4 Error demonstrations....30
9.5 Optimizer demonstration....31
9.6 Complex cases and transformation boundaries....32
9.7 All supplied example outcomes....33
9.8 Recorded performance evaluation....34
9.9 Performance charts and recorded observations....36
9.10 Limits of the evaluation....37
Chapter 10 Conclusion....38
10.1 Achievements against objectives....38
10.2 Limitations....38
10.3 Learning outcomes....38
Chapter 11 Future Enhancements....39
11.1 Language and validation extensions....39
11.2 Execution and optimization extensions....39

11.3 Evaluation extensions 39
References 40
Appendix A Final Grammar and Lexical Specification 41
A.1 Final EBNF 41
A.2 Lexical details 41
Appendix B Complete Diagnostic Catalogue 42
B.1 Lexical and syntax patterns 42
B.2 Semantic patterns and source locations 42
B.3 Executed semantic trigger specimens 44
B.4 Driver and dashboard boundaries 45
Appendix C Key Source Listings 46
C.1 AST data model 46
C.2 IR representation and lowering 47
C.3 Symbolic executor 48
C.4 Evaluation state snapshots used by the tests 48
Appendix D Example Programs and Sources 49
D.1 valid basic 50
D.2 valid drop 50
D.3 optimizer demo 50
D.4 complex multi model 51
Appendix E Reproduction and User Instructions 52
E.1 Compiler installation and CLI 52
E.2 Tests and stage inspection 52
E.3 Dashboard 52
E.4 Benchmark and report build 52
Appendix F Evolution Across Implementation Phases 54

List of Figures

Figure 5.1: Compiler architecture and representation flow 7
Figure 5.2: First-error control flow 8
Figure 5.3: Compiler package structure 9
Figure 7.1: AST produced for the basic valid pipeline 15
Figure 7.2: Symbol table evolution for the basic valid pipeline 18
Figure 9.1: Distribution of executed tests by file 29
Figure 9.2: Real original and optimized instruction sequences 31
Figure 9.3: Recorded stage-time breakdown by program size 36
Figure 9.4: Recorded microseconds per statement 36
Figure 9.5: Recorded original and optimized IR sizes 37

List of Tables

Table 2.1: Scope of the implemented processor 2
Table 3.1: Original objectives and final status 3
Table 4.1: Related approaches and the project boundary 5
Table 5.1: Compiler modules and responsibilities 8
Table 5.2: Dashboard pages and evidence shown 10
Table 6.1: Recorded development milestones 11
Table 7.1: Lexical patterns in scanner priority order 13
Table 7.2: AST node classes and declared fields 15
Table 7.3: SymbolTable fields 16
Table 7.4: Complete implemented semantic rule catalogue 17
Table 7.5: IR opcodes arguments and lowering sources 19
Table 7.6: Real symbolic trace examples by opcode 22
Table 7.7: Error classes and diagnostic formats 22
Table 8.1: Verified execution environment 23
Table 8.2: Compiler source line inventory 23
Table 8.3: CLI display flags 26
Table 9.1: Per-file executed tests and physical source lines 28
Table 9.2: Representative expected and actual outcomes 30
Table 9.3: Complete example-program outcomes 33
Table 9.4: Complete recorded benchmark results 35
Table A.1: Keyword to token mapping 41
Table B.1: All explicit lexical and syntax diagnostics 42
Table B.2: All explicit semantic diagnostics 42
Table D.1: Supplied example files and report roles 49
Table F.1: Phase 1 Phase 2 and final implementation evolution 54

List of Abbreviations

AST - Abstract Syntax Tree

CLI - Command-Line Interface

CRLF - Carriage Return and Line Feed

CSV - Comma-Separated Values

DAG - Directed Acyclic Graph

DSL - Domain-Specific Language

EBNF - Extended Backus-Naur Form

EOF - End of File

IR - Intermediate Representation

LF - Line Feed

LL(1) - Left-to-right predictive parsing with one token of lookahead

ML - Machine Learning

Chapter 1 Introduction

1.1 Background and problem area

A pipeline describes an ordered set of operations whose later steps depend on earlier declarations. Even a short workflow introduces relationships between a dataset, selected columns, named partitions and models. A textual description becomes easier to inspect when these relationships are represented as language rules rather than scattered across execution code.

Compiler construction provides a suitable separation of concerns. Lexical analysis recognizes the vocabulary, parsing establishes grammatical structure, and semantic analysis checks contextual relationships. A valid representation can then be lowered and transformed before an interpreter or another back end consumes it [1]. This project applies those stages to a restricted data science pipeline language.

1.2 Motivation

An ordering mistake such as training before defining a training partition can be identified without opening a dataset. Similarly, duplicate model names and percentages outside the accepted range are properties of the program description. Detecting these cases before a concrete runtime is invoked makes the source of a failure easier to isolate.

The project therefore concentrates on inspectable language processing. It does not measure time saved by avoiding real training jobs. The executor is deliberately symbolic, making demonstrations independent of dataset availability and machine learning libraries. The resulting traces expose instruction order without implying numerical computation.

1.3 Language and implementation overview

A source file contains one `pipeline` declaration. Its body may contain `load`, `clean`, `select`, `drop`, `split`, `train` and `evaluate` statements. The root declaration names the program; it is not an eighth executable body operation. The parser accepts an empty body, and semantics does not impose a final mandatory-load check on such a body.

The compiler package is `ds_pipeline_dsl`. It is driven by a command-line interface and reused by a Streamlit dashboard. Both expose actual compiler results. The dashboard adds presentation, stage timing and partial-result display without introducing new language statements.

1.4 Report organization

Chapters 2-4 establish the problem, objectives and background. Chapters 5-8 explain design, development, compiler concepts and implementation. Chapter 9 evaluates the results, followed by conclusions and future work. Appendices provide the grammar, diagnostics, listings, examples, reproduction commands and phase evolution.

Chapter 2 Problem Statement

2.1 Problem definition

The problem is to design and implement a small language processor that accepts a pipeline description, rejects malformed or contextually inconsistent programs with source-referenced diagnostics, and translates accepted programs into an inspectable instruction sequence. The processor must keep recognition, structural analysis, state validation and execution presentation separate.

Validation performed only after a concrete workflow begins can discover description errors later than necessary. This project demonstrates earlier detection for the specific invariants encoded in its semantic analyzer. It does not establish that general-purpose tools lack validation, nor does it provide a universal static analyzer for arbitrary Python programs.

2.2 Functional boundaries

Table 2.1 defines the implementation boundary against which the results are evaluated.

Table 2.1: Scope of the implemented processor

Area	Implemented scope	Boundary
Source language	One named pipeline with seven body statement forms.	No loops, expressions, functions or multiple pipeline blocks.
Validation	Token, grammar, declaration, ordering, column-state and split constraints.	No CSV schema lookup or general data-type inference.
Transformation	AST lowering and four IR optimizer passes.	No target machine-code generation or arbitrary control-flow analysis.
Execution	Readable symbolic trace from IR.	No file-content loading, model fitting, prediction or numeric accuracy.
Interaction	CLI and local Streamlit dashboard.	No distributed scheduler or visual node-based pipeline editor.

2.3 Inputs outputs and acceptance conditions

Input is source text, normally stored in a `.pipe` file. Successful processing produces tokens, an AST, a final symbol table, original IR, optimized IR, optimizer notes and trace strings. Expected compiler failures produce the original lexical, syntax or semantic diagnostic. The CLI returns zero for successful compilation and one for its handled compiler and file-reading errors. A request to display only one stage changes printed output, not the compiler stages that are executed.

Chapter 3 Objectives

3.1 Objectives carried forward from Phase 1

The original proposal specifies grammar design, tokenization, parsing, symbol-table state, semantic validation, IR lowering, optimization, symbolic execution and evaluation [12]. Those objectives remain the basis for the final assessment. Table 3.1 separates implemented dependencies and argument checks from broader type and scope checking that the initial wording suggested.

The objective-by-objective assessment is presented in Table 3.1; the original objective wording is retained.

Table 3.1: Original objectives and final status

No.	Phase 1 objective	Status	Evidence
1	Design a concise, readable, unambiguous grammar for describing data-processing pipelines, covering data loading, cleaning, feature selection, train/test splitting, model training, and evaluation.	Achieved	parser.py and grammar appendix
2	Implement a lexical analyzer that tokenizes the DSL's source files and reports lexical errors with line and column information.	Achieved	lexer.py and test_lexer.py
3	Implement a recursive-descent parser that validates the token stream against the grammar and constructs an Abstract Syntax Tree (AST), reporting syntax errors with clear diagnostics.	Achieved	parser.py and test_parser.py
4	Design and implement a symbol table that tracks the state of the pipeline (dataset loaded, columns selected, splits defined, models trained) as it is traversed.	Achieved	SymbolTable and test_semantic.py
5	Implement a semantic analyzer that performs dependency validation (e.g. detecting a 'train' stage that references an undefined split), scope checking, and type/argument checking, and produces meaningful diagnostic messages rather than raw error codes.	Partially achieved	analyze; scope and allowed-value limits in Section 3.2
6	Translate a semantically valid AST into a simple intermediate representation (Pipeline IR) suitable for optimization and execution.	Achieved	lower_to_ir and test_ir.py
7	Implement at least one optimization pass on the IR (e.g. elimination of redundant or unreachable pipeline stages).	Achieved	optimize and test_optimizer.py
8	Implement a symbolic execution engine that executes a validated IR and produces a readable execution trace, without requiring the project to become a machine-learning system.	Achieved	execute and test_executor.py

No.	Phase 1 objective	Status	Evidence
9	Evaluate the system against a suite of valid and deliberately invalid test pipelines covering lexical, syntactic, and semantic error classes.	Achieved	pytest log and all example CLI runs

3.2 Scope decisions and additional outcomes

Semantic dependency checking and argument constraints are implemented, including percentage bounds, duplicate names and selected/dropped-column relationships. Multiple pipeline scopes, a general type system and allowed-value lists for strategy or algorithm names were not pursued. Names such as a custom cleaning strategy remain valid identifiers. This limitation makes the semantic-analysis objective partial rather than implying implementation of a full type checker.

The final language adds `drop` beyond the Phase 1 grammar. The optimizer expands to four explicit passes, and the project includes a benchmark and dashboard. These additions extend the demonstration while retaining symbolic execution. ANTLR4 was considered in the proposal but was not adopted; lexical analysis and parsing remain hand-written.

Chapter 4 Literature Survey or Background Study

4.1 Workflow orchestration and library pipelines

Apache Airflow represents workflows as DAGs of tasks with dependencies and a scheduler/executor architecture [7]. Luigi supports dependency resolution and management of batch-task workflows [8]. Their task-level execution concerns are broader than the small source language implemented here. These systems should not be described as having no validation; this project instead makes its own statement-level semantic rules explicit.

Kubeflow Pipelines provides a platform for machine learning workflows [9]. Its pipeline tooling is distinct from this project, which neither deploys containers nor executes machine learning workloads. Scikit-learn Pipeline composes transformers and an optional final predictor through estimator interfaces [10]. It provides execution-oriented composition rather than the specific `.pipe` grammar, AST and source diagnostics implemented by this project.

Table 4.1 summarizes related approaches and the project boundary.

Table 4.1: Related approaches and the project boundary

Approach	Primary representation	Primary concern	Relation to this project
Airflow	DAGs and tasks	Workflow scheduling and execution.	Task orchestration is outside the implemented DSL.
Luigi	Tasks with dependencies	Batch-job dependency management.	Related dependency idea at a different abstraction level.
Kubeflow Pipelines	ML workflow components	Workflow compilation and execution infrastructure.	No deployment or concrete ML execution is provided here.
scikit-learn Pipeline	Transformers and estimators	Composable fit/transform/predict interfaces.	The present project validates a restricted textual description.
This project	Tokens, AST, SymbolTable and linear IR	Compiler-stage inspection and explicit static invariants.	Symbolic execution supports a reproducible compiler demonstration.

4.2 Compiler foundations

Lexical analysis groups characters into tokens using patterns. The Python `re` module supplies the compiled matching operations used by the scanner [3]. A parser then recognizes sequences of token types. Recursive descent is a top-down approach in which procedures follow grammar structure; it is suitable when the next token selects the required alternative without backtracking [2].

An AST stores semantic structure with less punctuation detail than the source. Dataclasses provide generated initialization and representation methods for the AST and IR objects [4]. A

symbol table records contextual information during a traversal. In this language it stores pipeline artifacts and definition lines rather than nested variable scopes.

An intermediate representation separates source structure from transformations and execution. The representation here is a linear list of opcode/argument records, not a register-based target language or a conventional three-address expression form. Optimization correctness is stated relative to the symbolic evaluation-record oracle implemented in the tests, rather than equality of every textual trace line.

4.3 Gap addressed

The project addresses the need for a small, inspectable example of compiler integration in a data science vocabulary. It exposes every representation, makes its invariants testable and produces deterministic traces for accepted examples. Its contribution is the combination of a restricted grammar, contextual validation, an explicit IR and checked transformations. It is not presented as a replacement for the execution platforms reviewed above.

Chapter 5 System Design

5.1 Architecture and data flow

Figure 5.1 shows the successful processing path. Each stage consumes a representation produced earlier in the pipeline.

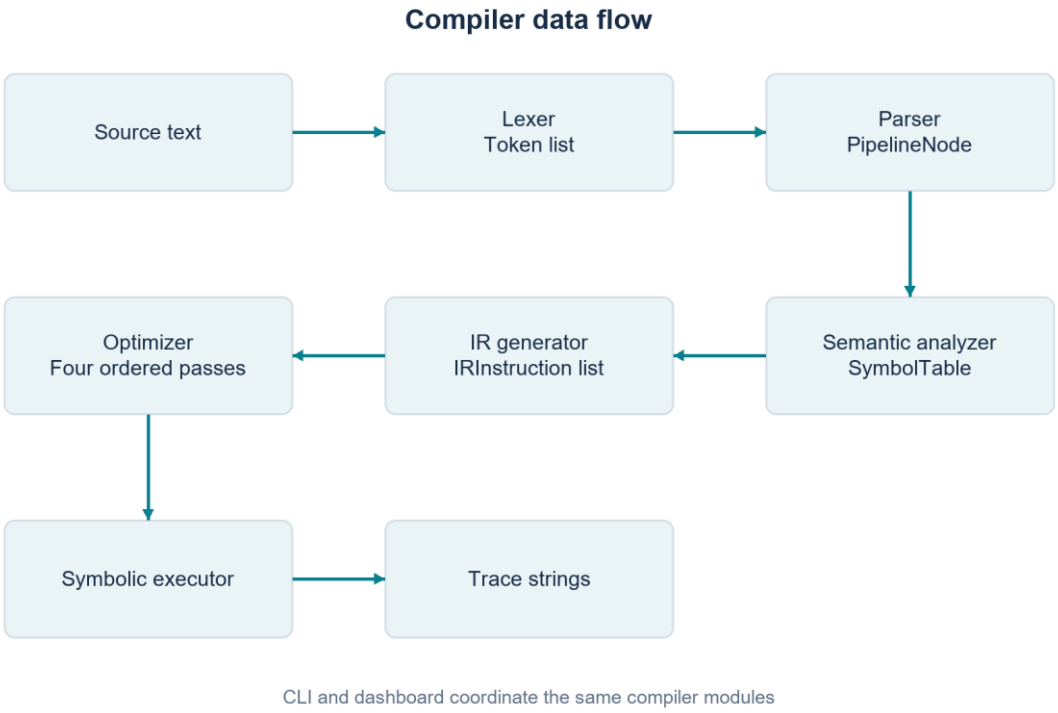


Figure 5.1: Compiler architecture and representation flow

The AST is retained while semantic analysis constructs a separate SymbolTable. Lowering receives both objects, although the current lowerer uses statement fields and does not consult the symbol-table parameter. Optimization returns a new instruction sequence and may append textual notes to an optional list. The executor returns trace strings rather than printing them.

5.2 Failure propagation

The three expected compiler error classes form the failure paths in Figure 5.2. A failure prevents later compiler stages from running.

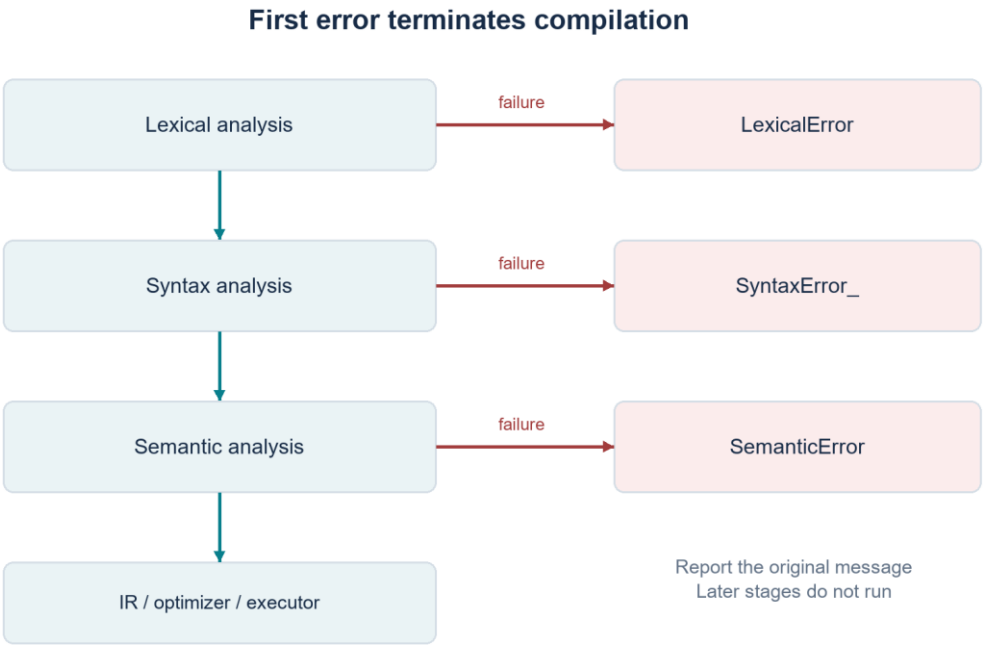


Figure 5.2: First-error control flow

The CLI catches the specific compiler exceptions and prints their messages. The dashboard adapter also catches unexpected exceptions, stores a traceback separately and marks subsequent stages skipped. Consequently, dashboard error resilience is broader than the CLI exception boundary; it does not change which programs the compiler accepts.

5.3 Module responsibilities

Table 5.1 summarizes compiler modules and responsibilities.

Table 5.1: Compiler modules and responsibilities

Module	Principal API	Responsibility
lexer.py	tokenize	Recognizes tokens and records source coordinates.
parser.py	Parser.parse_pipeline	Checks grammar and builds PipelineNode.
ast_nodes.py	PipelineNode and statement classes	Defines AST dataclasses.
semantic.py	analyze and SymbolTable	Checks dependencies and records symbolic state.
ir.py	IRInstruction and lower_to_ir	Lowers recognized statements into a flat instruction list.

Module	Principal API	Responsibility
optimizer.py	optimize	Applies four ordered IR transformations and records notes.
executor.py	execute	Formats symbolic trace entries from opcodes.
cli.py	main	Coordinates the stages, flags and expected compiler-error messages.

The module organization in Figure 5.3 corresponds to the physical package under `src/ds_pipeline_dsl/`.

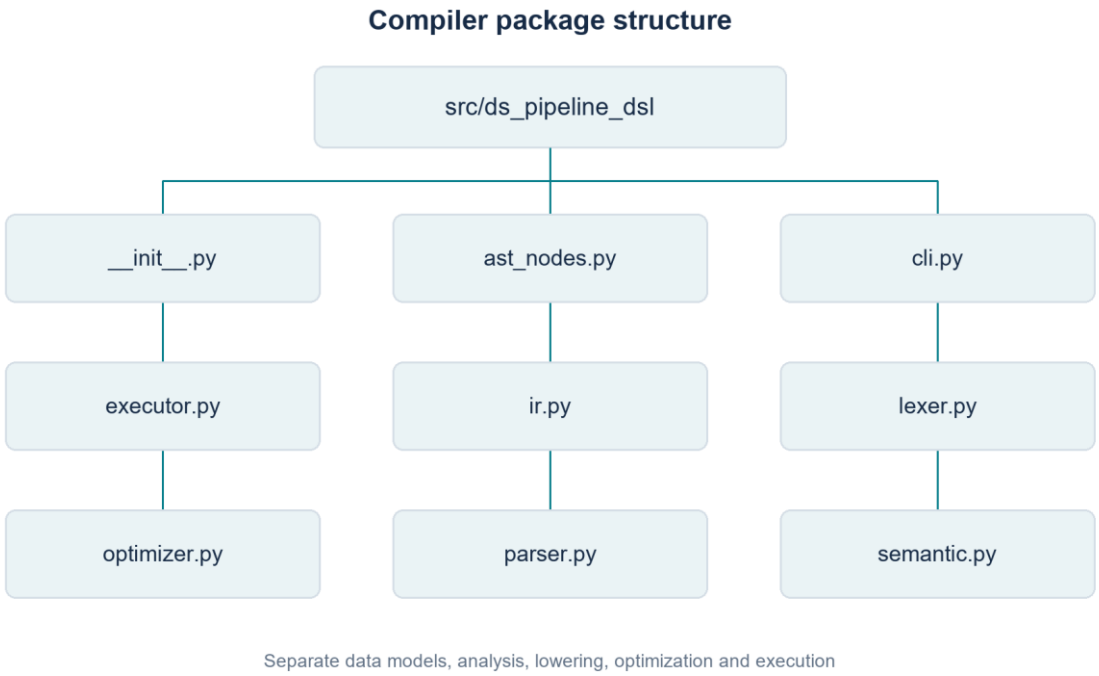


Figure 5.3: Compiler package structure

5.4 Symbol table and IR design

The symbol table models the state accumulated from earlier accepted statements. Definition-line maps make duplicate diagnostics explain both the current and previous declaration. `selected_columns=None` is meaningful: a drop before any selection is permitted after load, whereas a drop after selection must be drawn from the current selection and must leave at least one selected column. The complete field list appears in Table 7.3.

IRInstruction contains `op: str` and `args: dict`. Every recognized statement is lowered to one instruction in source order. Source lines are stored in the argument dictionary so the optimizer can explain removed or merged operations. The pipeline root itself has no executable opcode.

5.5 Interfaces and presentation design

The CLI uses argparse flags for tokens, AST, original IR, optimized IR and trace. With no display flag, all five display sections are printed, while semantic analysis runs between parsing and lowering without its own CLI section. This distinction explains why six processing stages appear in the dashboard but five headings appear in default CLI output.

Table 5.2 describes the implemented dashboard pages. It is an inspection interface over the compiler, not another language-processing implementation.

Table 5.2: Dashboard pages and evidence shown

Page	Presentation role
Overview	Static language label; grouped pipeline declaration and operations; discovered example count; saved test evidence; grammar reference.
Compiler Playground	Example selection, source editing, explicit Compile action, statuses, error highlighting, tokens, AST, semantic state, IR, optimization and trace.
Architecture	The compiler diagram, last-run colors, stage descriptions, implementation references and inline live counts.
Performance	Recorded environment, benchmark table, stage timing, time-per-statement and IR-size charts, and observations.

5.6 Project directory organization

```
src/ds_pipeline_dsl/  Compiler modules
tests/                Automated tests and semantic oracle
examples/             Supplied .pipe programs
benchmarks/           Synthetic benchmark driver
dashboard/            Streamlit UI and pure adapters
.streamlit/           Local dashboard configuration
docs/                 Earlier reports and project documentation
final_report/         Final report, build code, assets and evidence
```

Chapter 6 Methodology

6.1 Phased development

Phase 1 established the problem, grammar, proposed architecture and a working lexer/parser/AST/semantic prototype [12]. Phase 2 reorganized the implementation into a package and introduced IR, optimization, symbolic execution, CLI access and an automated suite documented as nineteen tests [13]. These phase reports describe intermediate states and are not used as the final API specification.

The final phase adds frontend and semantic hardening, the `drop` construct, safe optimizer passes with diagnostics, more tests, benchmarking and documentation. The current working tree also contains the Streamlit dashboard. Its presence is verified from files and tests; it is not attributed to a commit when the recorded history does not contain that dashboard addition.

6.2 Development history

The local Git history contains 19 recorded commits, including one merge commit. The sequence provides evidence of development themes and milestones, rather than a complete record of individual editing sessions. Table 6.1 summarizes those themes.

Table 6.1 summarizes recorded development milestones.

Table 6.1: Recorded development milestones

Commit or range	Verified theme
7e8c390 to e39721e	Initial repository, Phase 1 prototype/design and a merge.
4c8e14f to e0ed13e	Package refactoring, obsolete-prototype removal and split-range validation.
460041f to 90fe669	IR, an initial optimizer, symbolic executor and CLI.
ab9fe03 to 7fb8b1d	Phase 2 implementation/documentation and Review 2 presentation.
4e3dd75 and ac9564b	Final frontend/semantic improvements and complete drop integration.
5d490ed and e7d2cfe	Safe optimization diagnostics and comprehensive compiler/integration tests.
45a0532 and 9eca51a	Benchmarking, evaluation and final-phase documentation.

6.3 Design decisions

Hand-written regular-expression scanning and recursive-descent parsing keep the grammar-to-code relationship explicit. The body alternatives begin with distinct keyword tokens, and optional list repetition is selected by commas. A generated parser was unnecessary for this grammar, so the proposed ANTLR4 alternative was not pursued.

One pipeline per file avoids scope resolution across separate programs. Strategy and algorithm names remain free identifiers to avoid coupling syntax to a particular ML library.

First-error reporting limits cascading diagnostics, while the symbolic executor keeps the demonstration independent of data and training dependencies. Each choice reduces the implementation boundary in a way that can be explained and tested.

6.4 Optimizer corrections and regression validation

The Phase 2 optimizer captured in commit ab9fe03 sets a flag after encountering an `OP_EVAL` and removes every subsequent instruction. This is unsafe for a legal sequence containing a second model and evaluation. The final optimizer searches for the last evaluation before trimming the tail. Current regression and sensitivity tests make the lost-second-evaluation case explicit.

The earlier duplicate comparison includes all instruction arguments. Since lowering includes the source line in those arguments, identical operations on different lines can compare unequal. The final duplicate pass excludes line metadata from the semantic comparison while retaining it in surviving instructions and notes.

Current tests exercise the corrected behaviors, compare each pass with an evaluation-record oracle and test repeated optimization. The available commit order does not establish whether tests were written before each implementation edit; the methodology is therefore described as regression-driven validation rather than an unsupported claim about test-first chronology.

6.5 Reconciliation of earlier descriptions

The final Token is a dataclass, despite the Phase 2 text referring to a namedtuple. The processor has dependency/state checks rather than general type inference or multiple-pipeline scope checking. An empty pipeline is accepted, so a mandatory single-load claim would be incorrect. Unused and trailing models may appear in the semantic symbol table because that table describes the original AST; optimization subsequently removes their IR instructions where permitted.

Chapter 7 Compiler Design Concepts Used

7.1 Lexical analysis

The scanner compiles the ordered TOKEN_SPEC alternatives into one master regular expression with named groups. `finditer` yields matches from left to right. Identifiers that exactly match a reserved word are converted to the corresponding uppercase keyword token. Keyword recognition therefore does not split a longer identifier such as `trainer` into a keyword and suffix.

Table 7.1 preserves the actual regular expressions and their priority order. STRING precedes BAD_STRING, so a closed literal is recognized before the unterminated-string case. PERCENT precedes NUMBER, allowing the percentage suffix to form one token. EOF is appended explicitly and is not a regex alternative.

Table 7.1 summarizes lexical patterns in scanner priority order.

Table 7.1: Lexical patterns in scanner priority order

Order	Name	Regular expression	Role
1	STRING	"[^"\n]*"	Quoted string literal
2	BAD_STRING	"[^"\n]*	Unterminated string detector
3	PERCENT	\d+(\.\d+)?%	Numeric percentage with suffix
4	NUMBER	\d+(\.\d+)?	Bare numeric literal
5	LBRACE	\{	Opening brace
6	RBRACE	\}	Closing brace
7	EQUALS	=	Assignment separator
8	SEMI	;	Statement terminator
9	COMMA	,	Column-list separator
10	IDENT	[A-Za-z_][A-Za-z0-9_]*	Identifier or keyword candidate
11	SKIP	[\t\r\n]+	Whitespace, silently skipped
12	COMMENT	//[^\n]*	Line comment, silently skipped
13	MISMATCH	.	Unexpected-character detector
Synthetic	EOF	Empty value; appended after scanning	End-of-file marker

The reserved words are `clean`, `drop`, `evaluate`, `load`, `pipeline`, `select`, `split`, `train`. Whitespace includes spaces, tabs, CR and LF. Line tracking counts LF characters; a CRLF sequence therefore advances one line. Comment text is ignored until LF. The identifier regex

is ASCII-based, while comments and strings can contain Unicode characters. The digit patterns use Python regular-expression `\d`, not a separately enforced ASCII digit alphabet.

The scanner initializes line numbering to one and calculates columns as offsets from the current line start, making columns zero-based. The appended EOF token uses the current line and column zero. Unterminated strings and unexpected characters raise LexicalError. These are lexer errors rather than parser guesses about an invalid token.

7.2 Syntax analysis

```
pipeline_decl    ::= 'pipeline' IDENT '{' statement* '}'
statement        ::= load_stmt
                   | clean_stmt
                   | select_stmt
                   | drop_stmt
                   | split_stmt
                   | train_stmt
                   | evaluate_stmt

load_stmt        ::= 'load' STRING ';'
clean_stmt       ::= 'clean' IDENT ';'
select_stmt      ::= 'select' IDENT (',' IDENT)* ';'
drop_stmt        ::= 'drop' IDENT (',' IDENT)* ';'
split_stmt       ::= 'split' split_name '=' PERCENT ';'
split_name       ::= IDENT | 'train'
train_stmt       ::= 'train' IDENT '=' IDENT ';'
evaluate_stmt    ::= 'evaluate' IDENT ';'
```

The grammar above is taken from the parser specification and checked against its statement branches. `parse_pipeline` requires PIPELINE, an identifier and an opening brace; it repeatedly parses statements until a closing brace is found. It then requires the remaining token to be EOF, with a specific diagnostic for a second pipeline declaration.

The body alternatives have pairwise-disjoint FIRST sets: LOAD, CLEAN, SELECT, DROP, SPLIT, TRAIN and EVALUATE. The repeated statement sequence stops on RBRACE. A column-list continuation is selected by COMMA and otherwise the parser expects SEMI. These choices require only one lookahead token and no backtracking. There is no left-recursive production. This is the relevant LL(1) justification for the implemented grammar, rather than a claim about arbitrary grammar extensions.

The split-name position accepts IDENT or TRAIN. The word `test` is already an ordinary identifier, so it needs no separate reserved-token alternative. Other reserved words are not accepted where a normal identifier is expected. Percentages are tokenized with the suffix present; a bare NUMBER at a split value leads to an expected-PERCENT syntax diagnostic.

The parser uses `peek`, `advance` and `expect` to manage its token position. `expect` reports the required token type and the type/value found at the current token. A missing semicolon is often detected at the following statement token, so its error line can be later than the line where the omission occurred. This behavior is demonstrated by the supplied syntax-error example in Chapter 9.

7.3 Abstract syntax tree

Table 7.2 summarizes AST node classes and declared fields.

Table 7.2: AST node classes and declared fields

Node class	Fields
LoadStmt	filename: str; line: int
CleanStmt	strategy: str; line: int
SelectStmt	columns: List[str]; line: int
DropStmt	columns: List[str]; line: int
SplitStmt	name: str; percent: float; line: int
TrainStmt	model_name: str; algorithm: str; line: int
EvaluateStmt	model_name: str; line: int
PipelineNode	name: str; statements: List; line: int

Figure 7.1 shows the six statement nodes returned for `valid_basic.pipe`, retaining their actual fields and line numbers.

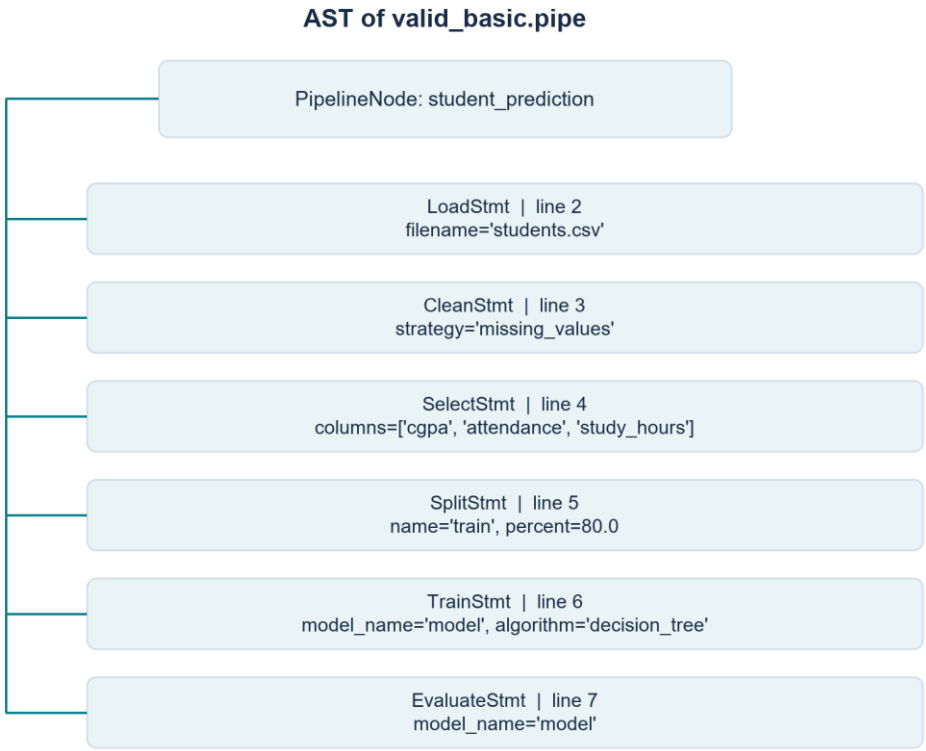


Figure 7.1: AST produced for the basic valid pipeline

The root stores the pipeline name and ordered statement list. Statement nodes keep the fields needed by later stages and the source line at which the keyword was encountered. The AST omits braces, commas and semicolons after they have served their parsing role. It therefore represents structure rather than a complete concrete syntax tree. Dataclass annotations document field intent but do not themselves implement runtime type enforcement.

7.4 Symbol table and semantic analysis

Table 7.3 lists all instance fields initialized by SymbolTable. They are populated by a left-to-right AST traversal.

Table 7.3: SymbolTable fields

Field	Stored form	Purpose
data_loaded	bool	Whether a load declaration has been accepted.
data_cleaned	bool	Whether at least one clean declaration has been accepted.
splits	dict	Split names mapped to percentages.
models	dict	Model names mapped to algorithm identifiers.
load_line	int or None	Source line of the accepted load declaration.
split_lines	dict	Definition lines of split names.
model_lines	dict	Definition lines of model names.
first_train	tuple or None	Name and line of the first accepted training declaration.
selected_columns	list or None	Current ordered selection; None means no select declaration yet.
dropped_columns	dict	Dropped column names mapped to their first drop lines.

The analyzer creates a fresh table for each pipeline. A successful load sets the dataset flag and declaration line. Clean updates its flag. Select replaces the active selection with a copied list. Drop remembers removed names and, when a selection exists, filters those names out of the active selection. Split inserts its name and percentage. Train records the model and algorithm; evaluate checks that the model has already been recorded.

The first training declaration also sets `first_train`. Clean, select and drop consult that field and are rejected after any model has been trained. A split after training remains legal if its own load, uniqueness and percentage conditions pass. Multiple evaluations are legal, including repeated evaluation of the same existing model. The analyzer does not require cleaning, selection, a test split or a final evaluation.

Table 7.4 covers every SemanticError raise site. Brace-delimited names are the actual interpolated Python expressions; Appendix B supplies a compiled trigger and real message for each specimen.

Table 7.4: Complete implemented semantic rule catalogue

Rule	Exact diagnostic pattern	Trigger specimen
S01 Duplicate load	Semantic Error at line {stmt.line}: 'load' used more than once; a dataset was already loaded at line {table.load_line}.	S01 in Appendix B
S02 Empty load filename	Semantic Error at line {stmt.line}: 'load' requires a non-empty filename.	S02 in Appendix B
S03 Clean before load	Semantic Error at line {stmt.line}: 'clean' used before any 'load' statement. No dataset is available.	S03 in Appendix B
S04 Clean after training	Semantic Error at line {stmt.line}: 'clean' used after model '{m}' was trained at line {t}; data-preparation statements must appear before any 'train'.	S04 in Appendix B
S05 Select before load	Semantic Error at line {stmt.line}: 'select' used before any 'load' statement.	S05 in Appendix B
S06 Select after training	Semantic Error at line {stmt.line}: 'select' used after model '{m}' was trained at line {t}; data-preparation statements must appear before any 'train'.	S06 in Appendix B
S07 Duplicate selected column	Semantic Error at line {stmt.line}: column '{col}' appears more than once in 'select'.	S07 in Appendix B
S08 Select previously dropped column	Semantic Error at line {stmt.line}: column '{col}' was dropped at line {m} and cannot be selected.	S08 in Appendix B
S09 Drop before load	Semantic Error at line {stmt.line}: 'drop' used before any 'load' statement.	S09 in Appendix B
S10 Drop after training	Semantic Error at line {stmt.line}: 'drop' used after model '{m}' was trained at line {t}; data-preparation statements must appear before any 'train'.	S10 in Appendix B
S11 Duplicate column in one drop	Semantic Error at line {stmt.line}: column '{col}' appears more than once in 'drop'.	S11 in Appendix B
S12 Drop previously dropped column	Semantic Error at line {stmt.line}: column '{col}' was already dropped at line {m}.	S12 in Appendix B
S13 Drop outside current selection	Semantic Error at line {stmt.line}: cannot drop column '{col}': it is not in the current selection.	S13 in Appendix B
S14 Drop every selected column	Semantic Error at line {stmt.line}: 'drop' would remove every selected column.	S14 in Appendix B
S15 Split before load	Semantic Error at line {stmt.line}: 'split' used before any 'load' statement.	S15 in Appendix B

Rule	Exact diagnostic pattern	Trigger specimen
S16 Split percentage outside bounds	Semantic Error at line {stmt.line}: split percentage must be strictly between 0 and 100.	S16 in Appendix B
S17 Duplicate split name	Semantic Error at line {stmt.line}: split '{stmt.name}' is already defined at line {m}.	S17 in Appendix B
S18 Cumulative split total over limit	Semantic Error at line {stmt.line}: cumulative split total would be {total:g}%, which exceeds 100%.	S18 in Appendix B
S19 Train without train split	Semantic Error at line {stmt.line}: training data has not been defined. A 'split train = <percent>%;' statement must appear before 'train {stmt.model_name} = {stmt.algorithm};'.	S19 in Appendix B
S20 Duplicate model name	Semantic Error at line {stmt.line}: model '{stmt.model_name}' is already defined at line {m}.	S20 in Appendix B
S21 Evaluation before training	Semantic Error at line {stmt.line}: model '{stmt.model_name}' is evaluated before it has been trained.	S21 in Appendix B

Each split percentage must be strictly between zero and one hundred. The cumulative comparison uses `total > 100 + 1e-9`, a small tolerance in the actual code. Duplicate split names are rejected before insertion. Column checks concern declarations within the source: before any select, a drop may introduce arbitrary column names after load; after select, dropped names must belong to the current selection. No physical dataset schema is consulted.

Figure 7.2 is computed by analyzing successive prefixes of the same real AST, rather than simulating the analyzer in report code.

Symbol table evolution for valid_basic.pipe

After statement	Loaded	Cleaned	Splits	Models
0 Initial	False	False	{}	{}
1 Load	True	False	{}	{}
2 Clean	True	True	{}	{}
3 Select	True	True	{}	{}
4 Split	True	True	{'train': 80.0}	{}
5 Train	True	True	{'train': 80.0}	{'model': 'decision_tree'}
6 Evaluate	True	True	{'train': 80.0}	{'model': 'decision_tree'}

Figure 7.2: Symbol table evolution for the basic valid pipeline

After the basic pipeline is analyzed, selected_columns contains cgpa, attendance and study_hours, splits contains train at 80.0, and models associates model with decision_tree. The table describes the original accepted AST. It is not rewritten after optimization, which explains why an unused model can remain in semantic state while its training instruction disappears from optimized IR.

7.5 Intermediate representation

Lowering maps each recognized statement to one IRInstruction and preserves statement order. The current lower_to_ir API accepts a SymbolTable parameter but does not read it; semantic validation is enforced by the driver before lowering. The records are domain opcodes with dictionaries, not conventional arithmetic three-address code.

Table 7.5 summarizes IR opcodes arguments and lowering sources.

Table 7.5: IR opcodes arguments and lowering sources

Opcode	Argument keys	AST source
OP_LOAD	filename, line	LoadStmt
OP_CLEAN	strategy, line	CleanStmt
OP_SELECT	columns, line	SelectStmt
OP_DROP	columns, line	DropStmt
OP_SPLIT	name, percent, line	SplitStmt
OP_TRAIN	model_name, algorithm, line	TrainStmt
OP_EVAL	model_name, line	EvaluateStmt

For valid_basic.pipe, six body statements produce six IR records. The root pipeline name does not produce an instruction. Source-line metadata survives lowering; it is used in optimizer notes and is deliberately ignored only when comparing the semantic arguments of duplicate clean/select operations.

7.6 Four-pass optimization

The optimizer calls _remove_unreachable, _remove_dead_trains, _merge_adjacent_drops and _collapse_duplicates in that order. Its notes parameter is optional. With a supplied list, transformations append diagnostic strings; with notes=None, no diagnostic print is required. Surviving instructions receive new record objects and copied argument dictionaries. Nested values such as column lists are not a general deep-copy guarantee, although the existing transformations do not mutate those input lists.

7.6.1 Removal after the last evaluation

Pass 1 searches backward for the final OP_EVAL and keeps instructions through that position. Instructions in the remaining tail cannot influence a later evaluation because no later

evaluation exists. The term unreachable is used relative to the selected observable-evaluation model; the original symbolic executor could still log those instructions if given unoptimized IR.

In the optimizer demonstration, the following original instruction is in the removed tail:

```
IRInstruction(op='OP_TRAIN', args={'model_name': 'trailing_model', 'algorithm': 'svm',
'line': 12})

unreachable: removed OP_TRAIN (line 12) after last OP_EVAL
```

Truncating after the first evaluation would be unsafe. A later train/evaluate pair is legal, and the multi-model tests require its evaluation record to survive. If the input contains no `OP_EVAL`, this pass conservatively returns copies without trimming the sequence.

7.6.2 Elimination of unused model training

Pass 2 performs a backward scan. Evaluation adds a model name to the live set; a relevant training instruction is retained and consumes that liveness requirement. A training instruction with no later evaluation of its model is omitted. For valid programs, model names are unique, so training an unevaluated model has no effect on another model record.

```
IRInstruction(op='OP_TRAIN', args={'model_name': 'dead_model', 'algorithm':
'logistic_regression', 'line': 10})

dead-train: removed OP_TRAIN 'dead_model' (line 10); model is never evaluated
```

The pass is conservative when no evaluation exists. It also retains training instructions if a hand-built `OP_EVAL` lacks model_name, and keeps a hand-built training instruction with no model_name. These defensive IR cases are tested, but malformed IR is not an accepted source-language feature.

7.6.3 Merging adjacent column drops

Pass 3 identifies immediately adjacent runs of at least two `OP_DROP` instructions with list-valued columns. It collects ordered, deduplicated column names, retains the first instruction line and appends one merged instruction. Only consecutive drops are merged; a split or clean instruction between drops prevents this transformation.

Selected original instructions from the captured demonstration are shown below; other IR lines are omitted.

```
IRInstruction(op='OP_DROP', args={'columns': ['student_id'], 'line': 5})
IRInstruction(op='OP_DROP', args={'columns': ['name'], 'line': 6})
(output truncated)
```

The corresponding optimized instruction and real note are:

```
IRInstruction(op='OP_DROP', args={'columns': ['student_id', 'name'], 'line': 5})

drop-merge: merged 2 adjacent OP_DROP (lines 5, 6) into one
```

The safety argument uses the set of dropped columns and the absence of an intervening observable operation. Each valid source drop before training updates only symbolic preparation state. Combining adjacent updates preserves the state subsequently captured by training and evaluation.

7.6.4 Collapsing adjacent duplicates

Pass 4 collapses consecutive `OP_CLEAN` instructions with the same strategy and consecutive `OP_SELECT` instructions with the same ordered columns. The argument comparison excludes the line key. Distinct strategies, different column order and non-adjacent repetitions are preserved.

```
IRInstruction(op='OP_CLEAN', args={'strategy': 'missing_values', 'line': 3})
IRInstruction(op='OP_CLEAN', args={'strategy': 'missing_values', 'line': 4})
(output truncated)

duplicate: removed repeated OP_CLEAN (line 4)
```

Repeated identical selection is idempotent in the symbolic selection state. Repeated identical cleaning is treated as idempotent by the test oracle when immediately adjacent. This is a model assumption about the symbolic language; it is not a guarantee that every concrete cleaning algorithm in a future runtime would be idempotent.

7.6.5 Observable behavior and safety limits

The tests define an evaluation record as the tuple (model_name, algorithm, train_state, eval_state). A state snapshot contains cleaning history, selected columns, the dropped-column set and sorted split entries. Training stores its state snapshot; evaluation records both that stored state and current state. The comparison is therefore stronger than merely checking the final evaluation message.

All preparation statements precede the first train in semantically accepted source. Removing an unused train therefore cannot bring previously separated preparation operations together. Splits can occur after training, so their timing remains significant: the oracle records the state at training separately from the state at evaluation. Only the tail after the last evaluation is discarded.

Complete instruction traces need not be identical after optimization. A merged drop has different text and a removed train no longer has a trace line. Evaluation-record equality is the correctness target. Tests additionally check operation order, surviving evaluations, dropped-column coverage, input preservation and idempotence on the supplied equivalence programs.

If no evaluation exists, only the two removal passes are conservative. Adjacent-drop merging and duplicate collapse still run. The live no-evaluation specimen in the evidence reduces three instructions to two by collapsing a duplicate clean. Consequently, a blanket statement that optimize always leaves no-evaluation pipelines unchanged would be false.

7.7 Symbolic execution

The executor walks the supplied instruction list and appends a formatted string for each opcode. Integer-valued split percentages are displayed without a decimal suffix. Unknown opcodes yield an Unknown instruction message rather than a compiler exception. No data-state engine, file-content reader, estimator object or numerical metric is created by execute.

Table 7.6 summarizes real symbolic trace examples by opcode.

Table 7.6: Real symbolic trace examples by opcode

Opcode	Captured trace text
OP_LOAD	Loaded students.csv
OP_CLEAN	Cleaned using missing_values
OP_SELECT	Selected columns: cgpa, attendance, study_hours
OP_DROP	Dropped columns: student_id, name
OP_SPLIT	Defined split 'train' = 80%
OP_TRAIN	Trained model 'model' using decision_tree
OP_EVAL	Evaluated model 'model' (simulated accuracy trace)

The existing evaluation string includes the phrase simulated accuracy trace. It is a label only: the output contains no measured accuracy value. Interpreting that label as a model-quality result would exceed the implementation.

7.8 Error detection and handling

Table 7.7 summarizes error classes and diagnostic formats.

Table 7.7: Error classes and diagnostic formats

Class	Raised by	Message format
LexicalError	lexer.py	Lexical Error at line L, col C: explanation
SyntaxError_	parser.py	Syntax Error at line L: explanation
SemanticError	semantic.py	Semantic Error at line L: explanation

The catalogue in Appendix B contains every explicit compiler diagnostic pattern extracted from raise statements. These errors stop compilation at the first detected violation. The CLI handles file-reading failures separately. The dashboard adds an Input result for oversized sources and an Internal result for unexpected exceptions, with technical details kept outside the main error view. Those wrapper categories are not new compiler exception classes or source-language rules.

Chapter 8 Implementation

8.1 Language tools and versions

Table 8.1 summarizes verified execution environment.

Table 8.1: Verified execution environment

Component	Recorded value
Project Python	3.14.7
Operating system	Windows-11-10.0.26200-SP0
pytest	9.1.1
Streamlit	1.66.0
Package metadata version	0.1.0
Compiler dependencies	Python standard library
Dashboard dependency declaration	streamlit>=1.66.0
Coverage	Not measured; pytest-cov not installed

Regular expressions, dataclasses, typing and argparse support the compiler. pytest supplies the test framework [5]. Streamlit supplies the optional dashboard [6], including pandas through its dependency set. Report-generation dependencies are confined to final_report and are not requirements of the compiler. Python 3.12 was the proposal platform; the current compiler evidence was collected using the Python version recorded above.

8.2 Source inventory

Table 8.2 counts physical and nonblank lines from the checked files. These measures include comments and docstrings and are not executable-statement counts.

Table 8.2: Compiler source line inventory

Module	Physical lines	Nonblank lines
__init__.py	4	4
ast_nodes.py	45	37
cli.py	87	75
executor.py	28	27
ir.py	31	29
lexer.py	68	59

Module	Physical lines	Nonblank lines
optimizer.py	157	136
parser.py	154	134
semantic.py	177	162

8.3 Core algorithms

The following pseudocode summarizes control flow; it is explanatory notation rather than a replacement implementation.

```
COMPILE(source)
  tokens <- tokenize(source)
  tree <- Parser(tokens).parse_pipeline()
  symbols <- analyze(tree)
  original <- lower_to_ir(tree, symbols)
  notes <- empty list
  optimized <- optimize(original, notes)
  trace <- execute(optimized)
  return completed representations
ON expected compiler error
  report original diagnostic and stop

ANALYZE(tree)
  create a fresh SymbolTable
  for each statement in source order
    check prerequisites and uniqueness for its statement form
    reject the first violated condition
    update the corresponding symbolic fields
  return SymbolTable

OPTIMIZE(instructions, notes)
  trim only the tail after the last evaluation
  scan backward to retain training needed by evaluations
  merge immediately adjacent drop runs
  collapse adjacent equal clean/select operations, ignoring line
  return the transformed instruction list
```

8.4 Selected implementation excerpts

The listings below are copied programmatically from the repository. Each excerpt contains at most twenty-five source lines. Their paths and ranges identify the original context; Appendix C provides longer supporting listings.

The Token data model stores the recognized type, literal value and both source coordinates. The lexical scanner creates these dataclass records.

Listing 8.1. `src/ds_pipeline_dsl/lexer.py`, lines 29-34.

```
@dataclass
class Token:
    type: str
    value: str
    line: int
    col: int
```

The parser expectation helper centralizes token mismatch diagnostics. It advances only after the expected token type is present.

Listing 8.2. `src/ds_pipeline_dsl/parser.py`, lines 53-60.

```
    def expect(self, ttype: str) -> Token:
        tok = self.peek()
        if tok.type != ttype:
            raise SyntaxError_(
                f"Syntax Error at line {tok.line}: expected {ttype}, "
                f"found {tok.type} ({tok.value!r})"
            )
        return self.advance()
```

SymbolTable initialization makes the analyzer state explicit. Flags, maps and definition lines are fresh for each call to analyze.

Listing 8.3. `src/ds_pipeline_dsl/semantic.py`, lines 13-23.

```
    def __init__(self):
        self.data_loaded = False
        self.data_cleaned = False
        self.splits: dict = {}
        self.models: dict = {}
        self.load_line = None
        self.split_lines: dict = {}
        self.model_lines: dict = {}
        self.first_train = None
        self.selected_columns = None
        self.dropped_columns: dict = {}
```

The optimizer entry point makes pass ordering visible. Optional notes are passed through to each transformation.

Listing 8.4. `src/ds_pipeline_dsl/optimizer.py`, lines 143-157.

```
def optimize(ir_list: List[IRInstruction], notes: Optional[list] = None) ->
List[IRInstruction]:
    """Optimizes the IR instructions through four independent passes:
    1. _remove_unreachable: removes dead tail instructions after the final evaluation.
    2. _remove_dead_trains: removes training of models that are never evaluated.
    3. _merge_adjacent_drops: combines runs of adjacent drop instructions.
    4. _collapse_duplicates: eliminates consecutive duplicate clean and select
instructions.

    Note on order: Order of passes is chosen for readability; the passes are independent
    because all data-preparation statements precede the first train (F1).
    """
    ir = _remove_unreachable(ir_list, notes)
    ir = _remove_dead_trains(ir, notes)
    ir = _merge_adjacent_drops(ir, notes)
    ir = _collapse_duplicates(ir, notes)
    return ir
```

The lowering routine dispatches on the dataclass type and emits one corresponding opcode. The executor dispatches on that opcode to produce a string. Neither routine performs general runtime type checking or concrete data science work. The CLI orchestrates these functions and chooses which results to print; the dashboard runner invokes the same APIs and retains stage results in a dataclass.

8.5 Command-line interface

Table 8.3 summarizes CLI display flags.

Table 8.3: CLI display flags

Flag	Printed representation
--tokens	Token records including EOF.
--ast	PipelineNode representation.
--ir	Original IR instructions.
--optimized-ir	Optimized IR, notes and instruction counts.
--trace	Symbolic trace.
No display flags	All five display sections.

```
python -m ds_pipeline_dsl.cli examples/valid_basic.pipe
python -m ds_pipeline_dsl.cli examples/optimizer_demo.pipe --ir --optimized-ir
python -m ds_pipeline_dsl.cli examples/complex_multi_model.pipe --trace
```

A display flag does not select a stopping stage: the driver continues through analysis, lowering, optimization and execution unless an error occurs. Expected compiler diagnostics are printed to standard output by the present CLI. argparse usage errors follow argparse behavior and are outside the three compiler error classes.

8.6 Benchmark implementation

The benchmark builds synthetic programs with repeated cleans, adjacent drops, repeated selects, evaluated and unevaluated model declarations, and one trailing training instruction. Its size parameter is not the final statement count. expected_counts derives the generated count, and correctness checks verify statement, IR and evaluation counts before timing.

The driver performs an untimed warm-up, collects garbage between repetitions, disables garbage collection inside timed regions and uses inner loops to reduce timer quantization effects. Three repetitions are configured, with per-stage medians reported. Total ms is the sum of the lexer, parser, semantic, IR, optimizer and optimized-execution medians; unoptimized execution is measured separately.

The safeguards bound the size parameter at 20000 and source length at 2000000 characters. A quadratic projection is checked against a 120-second overall budget. A repetition longer than 30 seconds can reduce repetitions after it completes; this is a post-run check, not a preemptive timeout that interrupts a hanging call. These limits are reported as implemented rather than described as stronger runtime guarantees.

8.7 Dashboard implementation

app.py contains presentation and routing. compiler_runner.py imports the compiler, captures stdout, times each real stage and retains earlier representations when a later stage fails. An

editor change does not overwrite the previous result; a source hash supports the stale-result notice. The example callback updates editor state before the textarea is created.

The runner rejects source text longer than 100000 characters before invoking the compiler. Empty text is passed through the real lexer and parser and produces a syntax error. Token display is capped at 1000 rows, with final EOF information retained for longer streams. Optimizer alignment labels are explicitly best-effort, while counts and notes come from real results.

performance_loader.py parses the saved Markdown table by its headers and retains the Environment and Observations section text. The Performance page does not automatically benchmark. Missing reports produce an explanatory message. The dashboard escapes source and compiler text used in custom HTML and presents unexpected technical details within expanders.

Chapter 9 Testing and Results

9.1 Test strategy and executed results

Tests are organized around lexical recognition, parsing, semantic validation, lowering, optimization, execution, CLI integration, supplied examples, boundaries, benchmark construction and dashboard behavior. Parametrization expands multiple inputs into independently reported test cases. The per-file counts below are derived from the verbose execution log, rather than counting function definitions.

The recorded run completed with 267 passed, 0 failed and 0 skipped tests. The dashboard contributes 44 passing cases; the remaining passing cases cover the compiler, examples, integration and benchmark. Coverage was not measured because pytest-cov was not installed. A passing suite is evidence for the tested cases, not a proof of all possible program behaviors.

Table 9.1 summarizes per-file executed tests and physical source lines.

Table 9.1: Per-file executed tests and physical source lines

Test file	Lines	Passed	Failed	Skipped
test_benchmark.py	154	10	0	0
test_boundary.py	138	16	0	0
test_cli.py	107	5	0	0
test_dashboard.py	341	44	0	0
test_examples.py	169	37	0	0
test_executor.py	51	2	0	0
test_integration.py	232	15	0	0
test_ir.py	63	2	0	0
test_lexer.py	112	23	0	0
test_optimizer.py	649	53	0	0
test_parser.py	168	27	0	0
test_semantic.py	439	33	0	0
Total	-	267	0	0

Figure 9.1 visualizes the same per-file counts. Optimizer and semantic tests receive substantial attention because transformations and contextual invariants require negative as well as positive cases.

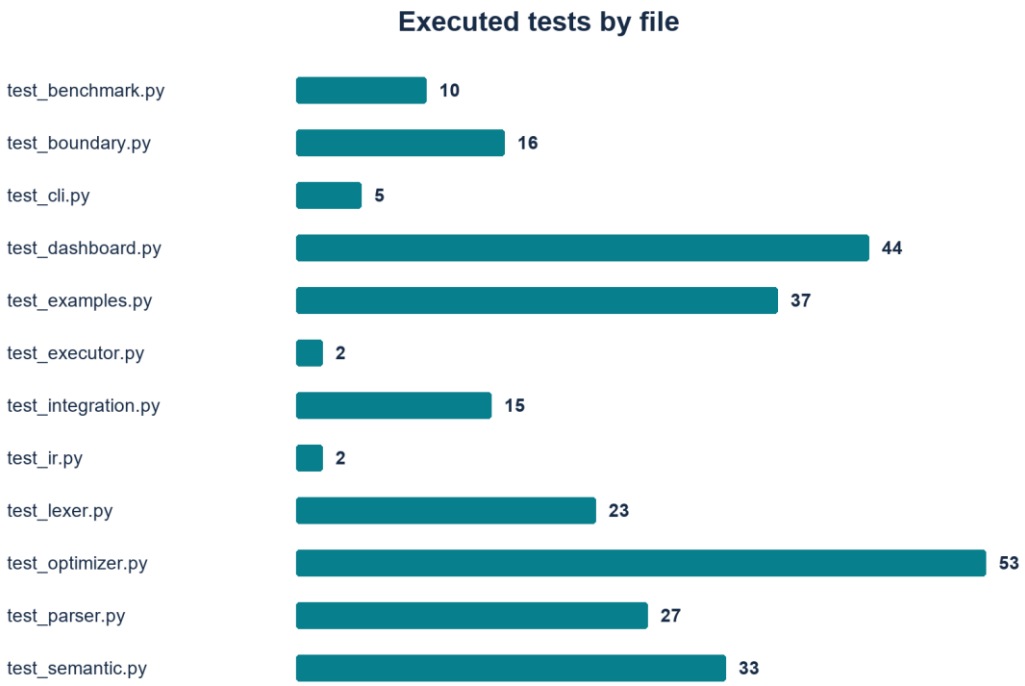


Figure 9.1: Distribution of executed tests by file

9.2 Baseline compilation and drop integration

The basic example contains 6 statements and produces 31 tokens including EOF. Lowering and optimization each return 6 instructions. Its complete optimized trace, captured by the CLI, is shown below.

```
Loaded students.csv
Cleaned using missing_values
Selected columns: cgpa, attendance, study_hours
Defined split 'train' = 80%
Trained model 'model' using decision_tree
Evaluated model 'model' (simulated accuracy trace)
```

The drop example is accepted and lowers DropStmt into `OP_DROP`. Its complete optimized trace confirms that the declared exclusion appears in the instruction log before selection. This checks integration through the lexer, AST, semantic state, IR and executor rather than only checking token recognition.

```
Loaded students.csv
Cleaned using missing_values
Dropped columns: student_id, name
Selected columns: cgpa, attendance, study_hours
Defined split 'train' = 80%
Trained model 'model' using decision_tree
Evaluated model 'model' (simulated accuracy trace)
```

9.3 Representative expected and actual cases

Table 9.2 compares intended cases with the final line of each actual CLI run. Full output and exit codes are retained in the evidence directory.

Table 9.2: Representative expected and actual outcomes

Input file	Expected outcome	Actual captured final line
valid_basic.pipe	Compile and produce an evaluation trace.	Evaluated model 'model' (simulated accuracy trace)
invalid_lexical_error.pipe	Reject unexpected @ in the lexer.	Lexical Error at line 2, col 9: unexpected character '@'
invalid_syntax_error.pipe	Reject the missing semicolon in the parser.	Syntax Error at line 3: expected SEMI, found CLEAN ('clean')
invalid_train_before_split.pipe	Reject training before train split.	Semantic Error at line 5: training data has not been defined. A 'split train = <percent>%;' statement must appear before 'train model = decision_tree;'.
invalid_split_total_over_100.pipe	Reject cumulative total above 100%.	Semantic Error at line 6: cumulative split total would be 110%, which exceeds 100%.
invalid_duplicate_model.pipe	Reject duplicate model name.	Semantic Error at line 7: model 'model' is already defined at line 6.
invalid_select_dropped_column.pipe	Reject selecting a previously dropped column.	Semantic Error at line 5: column 'student_id' was dropped at line 4 and cannot be selected.

9.4 Error demonstrations

`invalid_lexical_error.pipe` returns exit code 1. The illegal @ is reported at line 2, column 9. No token list is returned because tokenization fails before the driver reaches its token-printing section.

```
Lexical Error at line 2, col 9: unexpected character '@'
```

`invalid_syntax_error.pipe` returns exit code 1. The missing semicolon belongs to the load line, but the parser discovers the mismatch at the following CLEAN token on line 3. Lexical analysis has already succeeded.

```
Syntax Error at line 3: expected SEMI, found CLEAN ('clean')
```

`invalid_train_before_split.pipe` returns exit code 1. The AST is syntactically valid, but the training declaration occurs before the required train partition. Lowering and execution are not reached.

```
Semantic Error at line 5: training data has not been defined. A 'split train = <percent>%;'
statement must appear before 'train model = decision_tree;'.
```

`invalid_split_total_over_100.pipe` returns exit code 1. The analyzer adds the proposed split to the already declared percentages and rejects the resulting cumulative total.

```
Semantic Error at line 6: cumulative split total would be 110%, which exceeds 100%.
```

`invalid_drop_all_selected.pipe` returns exit code 1. Dropping every currently selected column is rejected before the symbolic state is updated.

```
Semantic Error at line 5: 'drop' would remove every selected column.
```

The dashboard tests verify these failure paths with source highlighting and stage statuses. For parser failures, tokens remain available in the wrapper result; for semantic failures, both tokens and AST remain available. The error message is preserved verbatim rather than reconstructed by the UI.

9.5 Optimizer demonstration

The optimizer demonstration lowers to 11 instructions and optimizes to 7 instructions, removing 4 instruction records. All four passes emit notes. The evaluation-record oracle returns equal original and optimized records for this program.

Figure 9.2 compares opcode order and source lines from the actual optimizer demonstration. The complete argument-bearing records are retained below and in the raw CLI evidence.

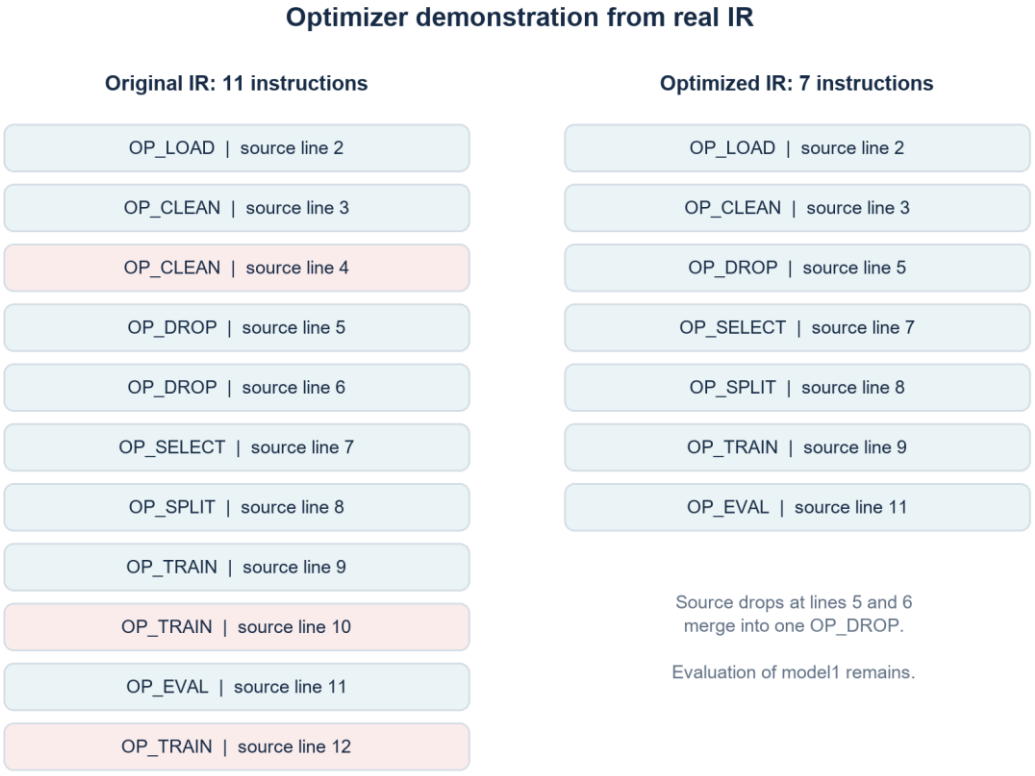


Figure 9.2: Real original and optimized instruction sequences

Original IR, copied from the captured CLI section:

```
IRInstruction(op='OP_LOAD', args={'filename': 'students.csv', 'line': 2})
IRInstruction(op='OP_CLEAN', args={'strategy': 'missing_values', 'line': 3})
IRInstruction(op='OP_CLEAN', args={'strategy': 'missing_values', 'line': 4})
IRInstruction(op='OP_DROP', args={'columns': ['student_id'], 'line': 5})
IRInstruction(op='OP_DROP', args={'columns': ['name'], 'line': 6})
IRInstruction(op='OP_SELECT', args={'columns': ['cgpa', 'attendance', 'study_hours'], 'line':
7})
IRInstruction(op='OP_SPLIT', args={'name': 'train', 'percent': 80.0, 'line': 8})
IRInstruction(op='OP_TRAIN', args={'model_name': 'model1', 'algorithm': 'decision_tree',
'line': 9})
IRInstruction(op='OP_TRAIN', args={'model_name': 'dead_model', 'algorithm':
'logistic_regression', 'line': 10})
IRInstruction(op='OP_EVAL', args={'model_name': 'model1', 'line': 11})
IRInstruction(op='OP_TRAIN', args={'model_name': 'trailing_model', 'algorithm': 'svm',
'line': 12})
```

Optimized IR, copied from the captured CLI section:

```
IRInstruction(op='OP_LOAD', args={'filename': 'students.csv', 'line': 2})
IRInstruction(op='OP_CLEAN', args={'strategy': 'missing_values', 'line': 3})
IRInstruction(op='OP_DROP', args={'columns': ['student_id', 'name'], 'line': 5})
IRInstruction(op='OP_SELECT', args={'columns': ['cgpa', 'attendance', 'study_hours'], 'line':
7})
IRInstruction(op='OP_SPLIT', args={'name': 'train', 'percent': 80.0, 'line': 8})
IRInstruction(op='OP_TRAIN', args={'model_name': 'model1', 'algorithm': 'decision_tree',
'line': 9})
IRInstruction(op='OP_EVAL', args={'model_name': 'model1', 'line': 11})
```

Optimizer notes and counts, copied from the captured CLI section:

```
unreachable: removed OP_TRAIN (line 12) after last OP_EVAL
dead-train: removed OP_TRAIN 'dead_model' (line 10); model is never evaluated
drop-merge: merged 2 adjacent OP_DROP (lines 5, 6) into one
duplicate: removed repeated OP_CLEAN (line 4)
IR instructions: 11 -> 7
```

9.6 Complex cases and transformation boundaries

The complex multi-model example reduces from 10 to 9 IR instructions. The unevaluated m_dead instruction is removed while both m1 and m2 evaluations survive. The actual optimized trace is:

```
Loaded students.csv
Dropped columns: id, notes
Selected columns: cgpa, attendance, study_hours, exam_score
Defined split 'train' = 70%
Defined split 'test' = 30%
Trained model 'm1' using decision_tree
Evaluated model 'm1' (simulated accuracy trace)
Trained model 'm2' using random_forest
Evaluated model 'm2' (simulated accuracy trace)
```

complex_redundant.pipe reduces from 12 to 7 instructions and exercises repeated preparation, adjacent drops, a dead model and a trailing model. complex_valid_drop_chain.pipe keeps its drops separate because a split lies between them. These examples distinguish safe local rewrites from broad merging across meaningful operations.

The optimizer suite also uses sensitivity tests: deleting a live training instruction, deleting a split, truncating after the first evaluation, removing a non-adjacent clean, reordering distinct

cleans or deleting a drop must change the oracle records. This validates that the oracle can detect representative wrong transformations. Ten named equivalence programs are checked both with the full optimizer and with each pass separately, and optimization is repeated to test idempotence on those programs.

9.7 All supplied example outcomes

Table 9.3 lists all supplied .pipe files. Classification combines the real accepted/error outcome with the optimizer/complex filename categories.

Table 9.3: Complete example-program outcomes

Example file	Classification	Exit	Result
complex_multi_model.pipe	complex	0	Accepted
complex_redundant.pipe	complex	0	Accepted
complex_valid_drop_chain.pipe	complex	0	Accepted
invalid_clean_after_train.pipe	invalid (semantic)	1	SemanticError
invalid_drop_after_train.pipe	invalid (semantic)	1	SemanticError
invalid_drop_all_selected.pipe	invalid (semantic)	1	SemanticError
invalid_drop_before_load.pipe	invalid (semantic)	1	SemanticError
invalid_drop_twice.pipe	invalid (semantic)	1	SemanticError
invalid_duplicate_load.pipe	invalid (semantic)	1	SemanticError
invalid_duplicate_model.pipe	invalid (semantic)	1	SemanticError
invalid_duplicate_split.pipe	invalid (semantic)	1	SemanticError
invalid_lexical_error.pipe	invalid (lexical)	1	LexicalError
invalid_select_dropped_column.pipe	invalid (semantic)	1	SemanticError
invalid_split_total_over_100.pipe	invalid (semantic)	1	SemanticError
invalid_syntax_error.pipe	invalid (syntax)	1	SyntaxError_
invalid_train_before_split.pipe	invalid (semantic)	1	SemanticError
optimizer_demo.pipe	optimization	0	Accepted
valid_basic.pipe	valid	0	Accepted
valid_drop.pipe	valid	0	Accepted
valid_select_then_drop.pipe	valid	0	Accepted

The supplied set contains 7 accepted programs and 13 rejected programs. The rejected programs contain one lexical example, one syntax example and 11 semantic examples. The table is exhaustive for the source files discovered during evidence collection.

9.8 Recorded performance evaluation

Performance measurements are read from docs/performance.md without running or overwriting the benchmark. The report environment below is retained verbatim and is separate from the current test-run environment in Chapter 8.

```
- Python version: 3.14.7
- Platform: Windows (Windows-11-10.0.26200-SP0)
- Benchmark run timestamp: 2026-10-22 12:06:40
- Total wall-clock duration: 0.63 s
```

The complete recorded table is placed on the following landscape page so that every original column and numeric string remains readable. The Statements column reports generated program sizes; these are not the nominal size parameters passed to build_program. Total ms excludes the separately measured unoptimized executor time.

Table 9.4 reproduces the saved benchmark table without changing its numeric strings, column names or status entries.

Table 9.4: Complete recorded benchmark results

Statements	Tokens	Lexer ms	Parser ms	Semantic ms	IR ms	Optimizer ms	Exec(opt) ms	Exec(unopt) ms	Total ms	us/stmt	IR before	IR after	Reduction %	Status
13	66	0.060	0.012	0.004	0.005	0.015	0.001	0.002	0.098	7.54	13	8	38.5%	ok
99	428	0.421	0.080	0.029	0.046	0.085	0.010	0.016	0.671	6.78	99	60	39.4%	ok
999	4208	4.428	0.842	0.315	0.452	0.818	0.098	0.163	6.952	6.96	999	600	39.9%	ok
9999	42008	45.302	8.014	2.955	5.082	10.714	1.494	2.519	73.560	7.36	9999	6000	40.0%	ok

Source: docs/performance.md. Units are milliseconds for stage/total columns and microseconds per statement for us/stmt. IR sizes and token counts are counts. The wall-clock duration above includes benchmark-harness work and is not the same quantity as one row of Total ms.

9.9 Performance charts and recorded observations

Figure 9.3 uses only the six stage-time columns contributing to Total ms. Program sizes are categorical, preventing misleading spacing between the four measured workloads.

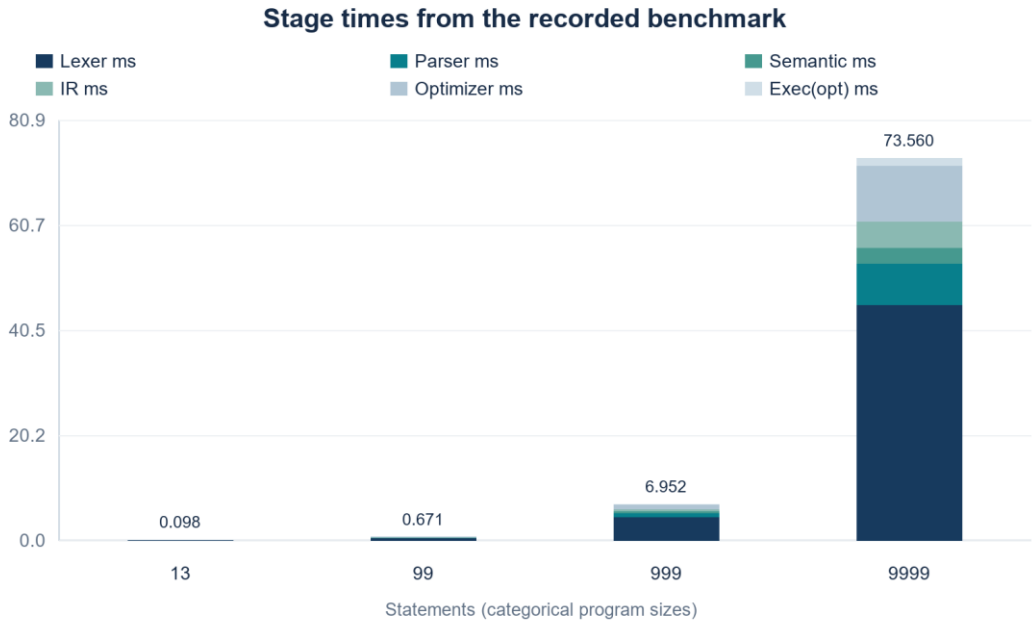


Figure 9.3: Recorded stage-time breakdown by program size

Figure 9.4 plots the us/stmt values exactly as recorded. Similar values for the larger template programs support an approximately linear trend for this workload, not a universal complexity proof.

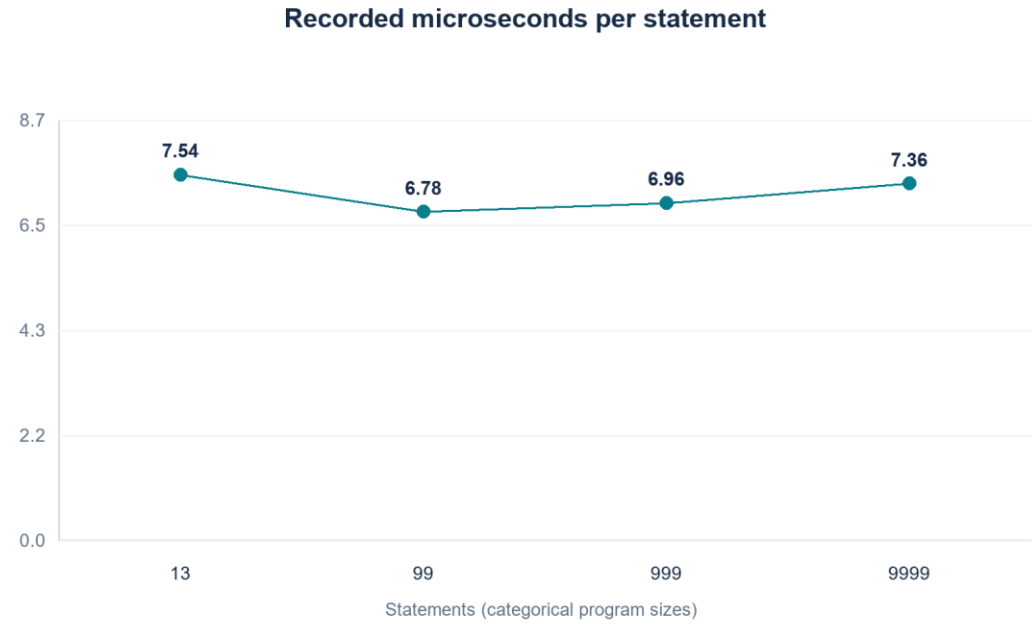


Figure 9.4: Recorded microseconds per statement

Figure 9.5 compares the saved IR before/after counts. The synthetic program deliberately includes patterns recognized by the optimizer.

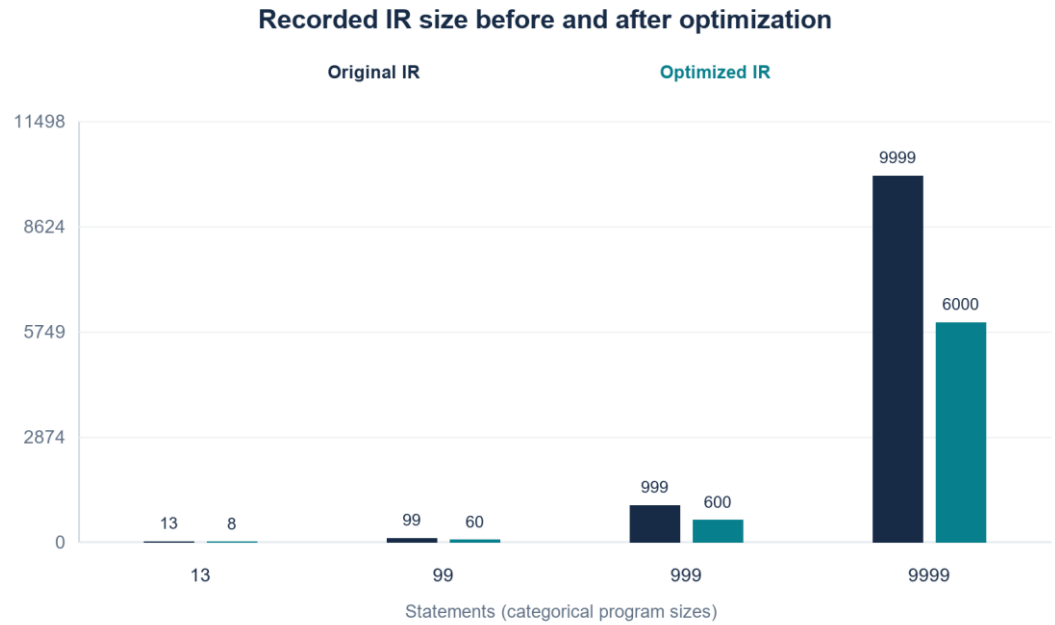


Figure 9.5: Recorded original and optimized IR sizes

The observations section of the saved benchmark report is reproduced verbatim below:

```
1. Scaling between 999 and 9999 statements: statement count grew by 10.01x while total
execution time grew by 10.58x.
2. Microseconds per statement ratio between 999 and 9999 statements is 1.06 (approximately
linear).
3. Dominant stage at 9999 statements: Lexer (45.302 ms, 61.6% of total pipeline time).
4. Execution speedup from optimization at 9999 statements: 1.69x (unoptimized 2.519 ms vs
optimized 1.494 ms).
5. Status: all configured benchmark sizes completed successfully with full repetitions.
```

9.10 Limits of the evaluation

The benchmark uses one machine, one generated template family and four recorded program sizes. Its approximately linear trend should not be generalized to every distribution of statements or column-list lengths. In particular, analyze recomputes the sum of all existing split percentages for each new split; many distinct small splits can increase that work beyond a constant amount per statement. Repeated selection/drop operations also manipulate lists whose sizes matter.

The recorded results contain no skipped or reduced-repetition rows. If later measurements contain them, their status must remain visible and numeric placeholders must not be interpreted as completed timings. No memory-profile measurement, concrete ML accuracy result or cross-machine comparison is supplied.

The dashboard tests exercise page execution and interaction through AppTest rather than a real browser layout audit. They establish that the UI calls the actual compiler and handles the tested states. They do not establish pixel-level rendering equivalence across browsers. Raw terminal evidence is supplied whether or not optional screenshots are available.

Chapter 10 Conclusion

10.1 Achievements against objectives

The completed system implements a restricted pipeline language through lexical analysis, predictive parsing, AST construction, symbol-table-based semantic checking, IR generation, four-pass optimization and symbolic execution. The CLI exposes intermediate representations and the dashboard makes the same processing stages accessible for interactive inspection.

The verified run records 267 passing tests and outcomes for 20 supplied examples. The optimizer demonstration produces eleven original instructions and seven optimized instructions while preserving its evaluation records. The recorded performance table provides stage timings and instruction counts for the generated workloads. These results meet the core language-processing and demonstration objectives.

10.2 Limitations

Execution is symbolic only. A filename declaration does not prove that a dataset exists or that named columns occur in a schema. Cleaning strategies and training algorithms are free identifiers. The implementation has no general static type system, no allowed-value catalogue and no concrete estimator backend.

Each source file contains one pipeline, and the compiler stops at its first expected error. It has no multi-error recovery, warning severity system, loops or user-defined functions. An empty pipeline is accepted. Optimizer safety is stated under the project test oracle and its symbolic preparation assumptions, not as a claim about arbitrary real data transformations.

The benchmark scope is limited to the recorded machine and synthetic template. A passing test suite does not establish exhaustive input coverage. These boundaries distinguish the achieved compiler demonstration from the future extensions listed in Chapter 11.

10.3 Learning outcomes

The implementation connects grammar design with parser control flow, declaration rules with state management, and transformations with explicit observable behavior. The optimizer evolution demonstrates why source metadata must be separated from semantic equality and why later evaluations cannot be discarded simply because an earlier evaluation exists. Exposing intermediate results makes those decisions inspectable during a technical review.

Chapter 11 Future Enhancements

11.1 Language and validation extensions

An optional schema could restrict cleaning strategies and model algorithms and describe required arguments. Such a change would require an explicit language-policy decision, new diagnostics and tests; it should not be implied by the current identifier grammar. Dataset-schema validation could be added separately with a clear distinction between source validation and external file inspection.

Multiple pipeline declarations would require a new program root, pipeline symbol scopes and rules for cross-pipeline references. Statement-level recovery could synchronize at semicolons or closing braces and collect more diagnostics, while avoiding updates from invalid declarations. A separate warning channel could expose unused models or redundant preparation without treating them as fatal errors.

11.2 Execution and optimization extensions

A scale statement is a plausible language extension, but it would need coordinated lexer, AST, parser, semantic, IR, executor and test changes. Its transformation properties would need to be defined before any optimizer could remove or reorder it.

A concrete execution backend could consume the existing IR through a separate runtime interface. It would need explicit dataset access, estimator dependencies, data partition semantics and error handling. The present symbolic idempotence assumptions would have to be reviewed for real algorithms, and equivalence tests would need observable results appropriate to that backend.

11.3 Evaluation extensions

Further evaluation could use additional program families, varied numbers of splits and columns, memory profiling, and multiple runtime environments. Property-based generation could complement the present example and boundary tests. Dashboard validation could add a supported browser-rendering check. These are prospective enhancements, not features or measurements claimed for the completed implementation.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson. [Online]. Available: https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003472/9780321486813. Accessed: 22 October 2026.

[2] R. Nystrom, Crafting Interpreters, chapter Parsing Expressions. [Online]. Available: https://craftinginterpreters.com/parsing-expressions.html. Accessed: 22 October 2026.

[3] Python Software Foundation, re - Regular expression operations, Python documentation. [Online]. Available: https://docs.python.org/3/library/re.html. Accessed: 22 October 2026.

[4] Python Software Foundation, dataclasses - Data Classes, Python documentation. [Online]. Available: https://docs.python.org/3/library/dataclasses.html. Accessed: 22 October 2026.

[5] pytest developers, pytest documentation. [Online]. Available: https://docs.pytest.org/en/stable/. Accessed: 22 October 2026.

[6] Streamlit, Streamlit documentation. [Online]. Available: https://docs.streamlit.io/. Accessed: 22 October 2026.

[7] Apache Software Foundation, Apache Airflow Architecture Overview. [Online]. Available: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/overview.html. Accessed: 22 October 2026.

[8] Luigi contributors, Getting Started, Luigi documentation. [Online]. Available: https://luigi.readthedocs.io/en/stable/. Accessed: 22 October 2026.

[9] Kubeflow contributors, Kubeflow Pipelines Overview. [Online]. Available: https://www.kubeflow.org/docs/components/pipelines/overview/. Accessed: 22 October 2026.

[10] scikit-learn developers, Pipeline, scikit-learn API documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline. Accessed: 22 October 2026.

[11] Compiler Design Laboratory Project Instruction Manual, supplied course document, section 24.

[12] S. B. Kurkuri, Phase 1 Problem Definition and System Design, BCSE307P, Fall Semester 2026-27. Supplied project report.

[13] S. B. Kurkuri, Phase 2 Core Implementation, BCSE307P, Fall Semester 2026-27. Supplied project report.

[14] S. B. Kurkuri, Data Science Pipeline DSL source repository and accompanying tests. Local working-tree evidence and recorded Git history. [Online]. Available: https://github.com/Sinchana-B-Kurkuri/A-Domain-Specific-Language-and-Compiler-Front-End-for-Data-Science-Pipelines. Accessed: 22 October 2026.

Appendix A Final Grammar and Lexical Specification

A.1 Final EBNF

The grammar is copied from parser.py. After the closing brace, the parser checks for EOF and rejects trailing input. TRAIN is contextual in split_name; test remains IDENT.

```
pipeline_decl    ::= 'pipeline' IDENT '{' statement* '}'
statement        ::= load_stmt
                   | clean_stmt
                   | select_stmt
                   | drop_stmt
                   | split_stmt
                   | train_stmt
                   | evaluate_stmt

load_stmt        ::= 'load' STRING ';'
clean_stmt       ::= 'clean' IDENT ';'
select_stmt      ::= 'select' IDENT (',' IDENT)* ';'
drop_stmt        ::= 'drop' IDENT (',' IDENT)* ';'
split_stmt       ::= 'split' split_name '=' PERCENT ';'
split_name       ::= IDENT | 'train'
train_stmt       ::= 'train' IDENT '=' IDENT ';'
evaluate_stmt    ::= 'evaluate' IDENT ';'
```

A.2 Lexical details

Table 7.1 gives regex priority. Keywords are case-sensitive; lines are one-based and columns zero-based. Whitespace accepts CRLF and LF. Comments extend from // to LF or EOF. Double-quoted strings exclude LF and have no escape-sequence decoding.

Table A.1 summarizes keyword to token mapping.

Table A.1: Keyword to token mapping

Keyword	Emitted token
clean	CLEAN
drop	DROP
evaluate	EVALUATE
load	LOAD
pipeline	PIPELINE
select	SELECT
split	SPLIT
train	TRAIN

The scanner appends Token("EOF", "", current_line, 0). Bare numbers are invalid split values. A negative percentage fails lexically at the unsupported minus character.

Appendix B Complete Diagnostic Catalogue

B.1 Lexical and syntax patterns

The following patterns are extracted from the current raise expressions. Brace-delimited expressions identify interpolated values, rather than missing report content. The source location column identifies the raise site in the compiler.

Table B.1 summarizes all explicit lexical and syntax diagnostics.

Table B.1: All explicit lexical and syntax diagnostics

Class and source	Exact message pattern
LexicalError lexer.py:54	Lexical Error at line {line_num}, col {col}: unterminated string literal
LexicalError lexer.py:58	Lexical Error at line {line_num}, col {col}: unexpected character {value!r}
SyntaxError_ parser.py:56	Syntax Error at line {tok.line}: expected {ttype}, found {tok.type} ({tok.value!r})
SyntaxError_ parser.py:70	Syntax Error at line {tok.line}: unexpected end of file; expected '}' to close pipeline '{name_tok.value}'
SyntaxError_ parser.py:78	Syntax Error at line {tok.line}: only one pipeline per file is supported
SyntaxError_ parser.py:82	Syntax Error at line {tok.line}: unexpected token {tok.type} ({tok.value!r}) after end of pipeline
SyntaxError_ parser.py:128	Syntax Error at line {name_tok.line}: expected split name, found {name_tok.type} ({name_tok.value!r})
SyntaxError_ parser.py:152	Syntax Error at line {tok.line}: unexpected token {tok.type} ({tok.value!r})

B.2 Semantic patterns and source locations

Table B.2 summarizes all explicit semantic diagnostics.

Table B.2: All explicit semantic diagnostics

Specimen and source	Exact message pattern
S01 semantic.py:30	Semantic Error at line {stmt.line}: 'load' used more than once; a dataset was already loaded at line {table.load_line}.
S02 semantic.py:35	Semantic Error at line {stmt.line}: 'load' requires a non-empty filename.
S03 semantic.py:43	Semantic Error at line {stmt.line}: 'clean' used before any 'load' statement. No dataset is available.

Specimen and source	Exact message pattern
S04 semantic.py:49	Semantic Error at line {stmt.line}: 'clean' used after model '{m}' was trained at line {t}; data-preparation statements must appear before any 'train'.
S05 semantic.py:57	Semantic Error at line {stmt.line}: 'select' used before any 'load' statement.
S06 semantic.py:63	Semantic Error at line {stmt.line}: 'select' used after model '{m}' was trained at line {t}; data-preparation statements must appear before any 'train'.
S07 semantic.py:70	Semantic Error at line {stmt.line}: column '{col}' appears more than once in 'select'.
S08 semantic.py:78	Semantic Error at line {stmt.line}: column '{col}' was dropped at line {m} and cannot be selected.
S09 semantic.py:86	Semantic Error at line {stmt.line}: 'drop' used before any 'load' statement.
S10 semantic.py:92	Semantic Error at line {stmt.line}: 'drop' used after model '{m}' was trained at line {t}; data-preparation statements must appear before any 'train'.
S11 semantic.py:99	Semantic Error at line {stmt.line}: column '{col}' appears more than once in 'drop'.
S12 semantic.py:107	Semantic Error at line {stmt.line}: column '{col}' was already dropped at line {m}.
S13 semantic.py:115	Semantic Error at line {stmt.line}: cannot drop column '{col}': it is not in the current selection.
S14 semantic.py:120	Semantic Error at line {stmt.line}: 'drop' would remove every selected column.
S15 semantic.py:133	Semantic Error at line {stmt.line}: 'split' used before any 'load' statement.
S16 semantic.py:138	Semantic Error at line {stmt.line}: split percentage must be strictly between 0 and 100.
S17 semantic.py:143	Semantic Error at line {stmt.line}: split '{stmt.name}' is already defined at line {m}.
S18 semantic.py:148	Semantic Error at line {stmt.line}: cumulative split total would be {total:g}%, which exceeds 100%.
S19 semantic.py:156	Semantic Error at line {stmt.line}: training data has not been defined. A 'split train = <percent>%;' statement must appear before 'train {stmt.model_name} = {stmt.algorithm};'.
S20 semantic.py:163	Semantic Error at line {stmt.line}: model '{stmt.model_name}' is already defined at line {m}.
S21 semantic.py:173	Semantic Error at line {stmt.line}: model '{stmt.model_name}' is evaluated before it has been trained.

B.3 Executed semantic trigger specimens

Each source below was passed to the real lexer, parser and analyzer during evidence extraction and raised SemanticError at the corresponding source-code raise site. These compact report specimens supplement, and do not modify, the existing test suite or examples. Their full sources and real messages are serialized in evidence/pipeline_data.json.

S01 Duplicate load

```
pipeline rule_case { load "a.csv"; load "b.csv"; }
```

```
Semantic Error at line 1: 'load' used more than once; a dataset was already loaded at line 1.
```

S02 Empty load filename

```
pipeline rule_case { load ""; }
```

```
Semantic Error at line 1: 'load' requires a non-empty filename.
```

S03 Clean before load

```
pipeline rule_case { clean missing; }
```

```
Semantic Error at line 1: 'clean' used before any 'load' statement. No dataset is available.
```

S04 Clean after training

```
pipeline rule_case { load "d.csv"; split train = 80%; train m = dt; clean missing; }
```

```
Semantic Error at line 1: 'clean' used after model 'm' was trained at line 1; data-
preparation statements must appear before any 'train'.
```

S05 Select before load

```
pipeline rule_case { select a; }
```

```
Semantic Error at line 1: 'select' used before any 'load' statement.
```

S06 Select after training

```
pipeline rule_case { load "d.csv"; split train = 80%; train m = dt; select a; }
```

```
Semantic Error at line 1: 'select' used after model 'm' was trained at line 1; data-
preparation statements must appear before any 'train'.
```

S07 Duplicate selected column

```
pipeline rule_case { load "d.csv"; select a, a; }
```

```
Semantic Error at line 1: column 'a' appears more than once in 'select'.
```

S08 Select previously dropped column

```
pipeline rule_case { load "d.csv"; drop a; select a; }
```

```
Semantic Error at line 1: column 'a' was dropped at line 1 and cannot be selected.
```

S09 Drop before load

```
pipeline rule_case { drop a; }
```

```
Semantic Error at line 1: 'drop' used before any 'load' statement.
```

S10 Drop after training

```
pipeline rule_case { load "d.csv"; split train = 80%; train m = dt; drop a; }
```

```
Semantic Error at line 1: 'drop' used after model 'm' was trained at line 1; data-preparation
statements must appear before any 'train'.
```

S11 Duplicate column in one drop

```
pipeline rule_case { load "d.csv"; drop a, a; }
```

```
Semantic Error at line 1: column 'a' appears more than once in 'drop'.
```

S12 Drop previously dropped column

```
pipeline rule_case { load "d.csv"; drop a; drop a; }
Semantic Error at line 1: column 'a' was already dropped at line 1.
```

S13 Drop outside current selection

```
pipeline rule_case { load "d.csv"; select a, b; drop c; }
Semantic Error at line 1: cannot drop column 'c': it is not in the current selection.
```

S14 Drop every selected column

```
pipeline rule_case { load "d.csv"; select a; drop a; }
Semantic Error at line 1: 'drop' would remove every selected column.
```

S15 Split before load

```
pipeline rule_case { split train = 80%; }
Semantic Error at line 1: 'split' used before any 'load' statement.
```

S16 Split percentage outside bounds

```
pipeline rule_case { load "d.csv"; split train = 100%; }
Semantic Error at line 1: split percentage must be strictly between 0 and 100.
```

S17 Duplicate split name

```
pipeline rule_case { load "d.csv"; split train = 80%; split train = 10%; }
Semantic Error at line 1: split 'train' is already defined at line 1.
```

S18 Cumulative split total over limit

```
pipeline rule_case { load "d.csv"; split train = 80%; split test = 30%; }
Semantic Error at line 1: cumulative split total would be 110%, which exceeds 100%.
```

S19 Train without train split

```
pipeline rule_case { load "d.csv"; train m = dt; }
Semantic Error at line 1: training data has not been defined. A 'split train = <percent>%;'
statement must appear before 'train m = dt;'.
```

S20 Duplicate model name

```
pipeline rule_case { load "d.csv"; split train = 80%; train m = dt; train m = rf; }
Semantic Error at line 1: model 'm' is already defined at line 1.
```

S21 Evaluation before training

```
pipeline rule_case { evaluate m; }
Semantic Error at line 1: model 'm' is evaluated before it has been trained.
```

B.4 Driver and dashboard boundaries

File-reading exceptions are caught by cli.py and prefixed with Error reading file. argparse manages command-line usage errors separately. The dashboard runner adds Input and Internal result categories, retaining unexpected tracebacks for an expander. These presentation-layer categories are not entries in the compiler lexical, syntax or semantic catalogue.

Appendix C Key Source Listings

The following listings are copied directly from the current source files. They complement the short excerpts in Chapter 8 and show how data models, lowering and execution connect. No line numbers are inserted into the source text.

C.1 AST data model

Verbatim listing: `src/ds_pipeline_dsl/ast_nodes.py`, lines 1-45.

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class LoadStmt:
    filename: str
    line: int

@dataclass
class CleanStmt:
    strategy: str
    line: int

@dataclass
class SelectStmt:
    columns: List[str]
    line: int

@dataclass
class DropStmt:
    columns: List[str]
    line: int

@dataclass
class SplitStmt:
    name: str
    percent: float
    line: int

@dataclass
class TrainStmt:
    model_name: str
    algorithm: str
    line: int

@dataclass
class EvaluateStmt:
    model_name: str
    line: int


@dataclass
class PipelineNode:
    name: str
    statements: List = field(default_factory=list)
    line: int = 0
```

C.2 IR representation and lowering

Verbatim listing: `src/ds_pipeline_dsl/ir.py`, lines 1-31.

```
from dataclasses import dataclass
from typing import List
from ds_pipeline_dsl.ast_nodes import (
    PipelineNode, LoadStmt, CleanStmt, SelectStmt, DropStmt,
    SplitStmt, TrainStmt, EvaluateStmt
)
from ds_pipeline_dsl.semantic import SymbolTable

@dataclass
class IRInstruction:
    op: str
    args: dict

def lower_to_ir(pipeline_ast: PipelineNode, symbol_table: SymbolTable) ->
List[IRInstruction]:
    ir = []
    for stmt in pipeline_ast.statements:
        if isinstance(stmt, LoadStmt):
            ir.append(IRInstruction(op="OP_LOAD", args={"filename": stmt.filename,
"line": stmt.line}))
        elif isinstance(stmt, CleanStmt):
            ir.append(IRInstruction(op="OP_CLEAN", args={"strategy": stmt.strategy,
"line": stmt.line}))
        elif isinstance(stmt, SelectStmt):
            ir.append(IRInstruction(op="OP_SELECT", args={"columns": stmt.columns,
"line": stmt.line}))
        elif isinstance(stmt, DropStmt):
            ir.append(IRInstruction(op="OP_DROP", args={"columns": stmt.columns, "line":
stmt.line}))
        elif isinstance(stmt, SplitStmt):
            ir.append(IRInstruction(op="OP_SPLIT", args={"name": stmt.name, "percent":
stmt.percent, "line": stmt.line}))
        elif isinstance(stmt, TrainStmt):
            ir.append(IRInstruction(op="OP_TRAIN", args={"model_name": stmt.model_name,
"algorithm": stmt.algorithm, "line": stmt.line}))
        elif isinstance(stmt, EvaluateStmt):
            ir.append(IRInstruction(op="OP_EVAL", args={"model_name": stmt.model_name,
"line": stmt.line}))
    return ir
```

C.3 Symbolic executor

Verbatim listing: `src/ds_pipeline_dsl/executor.py`, lines 1-28.

```
from typing import List
from ds_pipeline_dsl.ir import IRInstruction

def execute(ir_list: List[IRInstruction]) -> List[str]:
    trace = []
    for instr in ir_list:
        if instr.op == "OP_LOAD":
            trace.append(f"Loaded {instr.args['filename']}")
        elif instr.op == "OP_CLEAN":
            trace.append(f"Cleaned using {instr.args['strategy']}")
        elif instr.op == "OP_SELECT":
            cols = ", ".join(instr.args["columns"])
            trace.append(f"Selected columns: {cols}")
        elif instr.op == "OP_DROP":
            cols = ", ".join(instr.args["columns"])
            trace.append(f"Dropped columns: {cols}")
        elif instr.op == "OP_SPLIT":
            percent = instr.args["percent"]
            if percent == int(percent):
                percent = int(percent)
            trace.append(f"Defined split '{instr.args['name']}' = {percent}%")
        elif instr.op == "OP_TRAIN":
            trace.append(f"Trained model '{instr.args['model_name']}' using
{instr.args['algorithm']}")
        elif instr.op == "OP_EVAL":
            trace.append(f"Evaluated model '{instr.args['model_name']}' (simulated
accuracy trace)")
        else:
            trace.append(f"Unknown instruction: {instr.op}")
    return trace
```

C.4 Evaluation state snapshots used by the tests

Verbatim listing: `tests/helpers.py`, lines 13-22.

```
def _snapshot_state(cleaning_history: List[str],
                    selected_columns: Optional[Tuple[str, ...]],
                    dropped_columns: set,
                    splits: dict) -> Tuple[Tuple[str, ...], Optional[Tuple[str, ...]],
frozenset, Tuple[Tuple[str, float], ...]]:
    return (
        tuple(cleaning_history),
        selected_columns,
        frozenset(dropped_columns),
        tuple(sorted(splits.items()))
    )
```

The evaluation_records helper uses these snapshots at training and evaluation points. Its complete implementation remains in tests/helpers.py and is identified in the claims ledger. This state representation is the basis for the optimizer safety discussion in Section 7.6.5.

Appendix D Example Programs and Sources

Table D.1 indexes the complete supplied example set. Measured exit codes and outcomes appear in Table 9.3.

Table D.1: Supplied example files and report roles

Example file	Role
complex_multi_model.pipe	complex
complex_redundant.pipe	complex
complex_valid_drop_chain.pipe	complex
invalid_clean_after_train.pipe	invalid (semantic)
invalid_drop_after_train.pipe	invalid (semantic)
invalid_drop_all_selected.pipe	invalid (semantic)
invalid_drop_before_load.pipe	invalid (semantic)
invalid_drop_twice.pipe	invalid (semantic)
invalid_duplicate_load.pipe	invalid (semantic)
invalid_duplicate_model.pipe	invalid (semantic)
invalid_duplicate_split.pipe	invalid (semantic)
invalid_lexical_error.pipe	invalid (lexical)
invalid_select_dropped_column.pipe	invalid (semantic)
invalid_split_total_over_100.pipe	invalid (semantic)
invalid_syntax_error.pipe	invalid (syntax)
invalid_train_before_split.pipe	invalid (semantic)
optimizer_demo.pipe	optimization
valid_basic.pipe	valid
valid_drop.pipe	valid
valid_select_then_drop.pipe	valid

D.1 valid basic

Source: `examples/valid_basic.pipe`. The text below is copied verbatim.

```
pipeline student_prediction {
    load "students.csv";
    clean missing_values;
    select cgpa, attendance, study_hours;
    split train = 80%;
    train model = decision_tree;
    evaluate model;
}
```

D.2 valid drop

Source: `examples/valid_drop.pipe`. The text below is copied verbatim.

```
pipeline student_prediction {
    load "students.csv";
    clean missing_values;
    drop student_id, name;
    select cgpa, attendance, study_hours;
    split train = 80%;
    train model = decision_tree;
    evaluate model;
}
```

D.3 optimizer demo

Source: `examples/optimizer_demo.pipe`. The text below is copied verbatim.

```
pipeline student_prediction {
    load "students.csv";
    clean missing_values;
    clean missing_values;
    drop student_id;
    drop name;
    select cgpa, attendance, study_hours;
    split train = 80%;
    train model1 = decision_tree;
    train dead_model = logistic_regression;
    evaluate model1;
    train trailing_model = svm;
}
```

D.4 complex multi model

Source: `examples/complex_multi_model.pipe`. The text below is copied verbatim.

```
pipeline complex_multi_model {
    load "students.csv";
    drop id, notes;
    select cgpa, attendance, study_hours, exam_score;
    split train = 70%;
    split test = 30%;
    train m_dead = logistic_regression;
    train m1 = decision_tree;
    evaluate m1;
    train m2 = random_forest;
    evaluate m2;
}
```

Appendix E Reproduction and User Instructions

E.1 Compiler installation and CLI

Run the commands from the repository root in PowerShell. The compiler package itself has no third-party runtime dependency. pytest is required for the test command; Streamlit is required only for the dashboard.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e .
python -m pip install pytest
python -m ds_pipeline_dsl.cli examples/valid_basic.pipe
```

If PowerShell blocks activation, the process-scoped command below can be run before retrying activation. It does not make a permanent policy change.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

E.2 Tests and stage inspection

```
python -m pytest -v
python -m ds_pipeline_dsl.cli examples/valid_basic.pipe --tokens
python -m ds_pipeline_dsl.cli examples/valid_basic.pipe --ast
python -m ds_pipeline_dsl.cli examples/optimizer_demo.pipe --ir --optimized-ir
python -m ds_pipeline_dsl.cli examples/complex_multi_model.pipe --trace
```

Display flags select output sections but do not stop compilation early. Invalid source exits with code one for the handled compiler errors. The verbose test log used for this report is saved under final_report/evidence/pytest_v.txt; its cache-writing plugin was disabled during evidence collection.

E.3 Dashboard

```
python -m pip install -r requirements-dashboard.txt
python -m streamlit run dashboard/app.py
```

Open http://localhost:8501 and keep the server process running. If the default port is occupied, use the command below and open http://localhost:8502. The dashboard inserts the repository src directory into its import path and does not require CSV files to demonstrate symbolic traces.

```
python -m streamlit run dashboard/app.py --server.port 8502
```

Select an example, inspect its source and press Compile. Use the six tabs for representations, the Architecture page for last-run stage colors and the Performance page for the saved report. Editing source leaves previous results visible until compilation is requested again.

E.4 Benchmark and report build

```
python benchmarks/benchmark.py
```

The normal benchmark command writes docs/performance.md. It was not run while producing this report because an existing report was available. The report build instead copies that file into its evidence directory. If the saved report is absent, evidence collection invokes the benchmark functions directly and writes the new report only under final_report/evidence/.

```
python -m pip install -r final_report/requirements-report.txt
python final_report/build_report.py --project-python .venv/Scripts/python.exe
```

The report builder regenerates evidence, diagrams and the DOCX. On Windows with Microsoft Word installed it updates fields and exports PDF through Word automation; a configured LibreOffice executable can be used on supported environments. The project-python option selects the interpreter in which pytest and the optional dashboard are already installed. The report README records the verified absolute command for this machine.

Appendix F Evolution Across Implementation Phases

Table F.1 consolidates the documented phases into one implementation history. Proposal items are not treated as completed features without code evidence.

Table F.1: Phase 1 Phase 2 and final implementation evolution

Area	Phase 1 report	Phase 2 report	Final verified state
Language	load/clean/select/split/train/evaluate proposal.	Core statement forms retained.	Adds drop throughout all stages; one pipeline per file.
Organization	Single-file prototype and design.	Modular ds_pipeline_dsl package.	Compiler modules, benchmark, expanded tests and dashboard.
Tokens and AST	Typed token stream and AST prototype.	Text calls Token a namedtuple.	Token and AST nodes are dataclasses.
Semantic scope	Dependency, scope and type checking proposed.	Dependency checks and split bounds.	Twenty-one explicit state/declaration checks; no general types or multiple-pipeline scopes.
IR and executor	Planned in the proposal.	Linear IR and symbolic trace.	Seven opcodes; source-line metadata; symbolic execution retained.
Optimizer	At least one pass proposed.	Duplicate collapse and removal after an evaluation.	Four passes; final-evaluation boundary; metadata-aware equality; diagnostic notes.
Evaluation	Prototype valid/invalid demonstrations.	Nineteen tests and four examples recorded.	Current suite and all supplied examples measured in Chapter 9.
Presentation	Design document and initial review.	CLI and Review 2 materials.	CLI, dashboard, saved benchmark evidence and final report.

The final report retains the original project purpose while correcting obsolete implementation details. ANTLR4 remained an alternative, model and cleaning names remained free identifiers, and the symbolic execution boundary remained unchanged. Historical counts and example outputs are identified as phase-specific rather than reused as current measurements.